A Formal Security Framework for MCP-Based AI Agents: Threat Taxonomy, Verification Models, and Defense Mechanisms

Nirajan Acharya[1], Gaurav Kumar Gupta[2] [1]*University of the Cumberlands*
Email: nacharya39400@ucumberlands.edu [2]*Youngstown State University*
Email: guptagauravk1@gmail.com

arXiv:2604.05969v1 [cs.CR] 7 Apr 2026

***Abstract*—The Model Context Protocol (MCP), introduced by Anthropic in November 2024 and now governed by the Linux Foundation's Agentic AI Foundation, has rapidly become the de facto standard for connecting large language model (LLM)-based agents to external tools and data sources, with over 97 million monthly SDK downloads and more than 177,000 registered tools. However, this explosive adoption has exposed a critical gap: the absence of a unified, formal security framework capable of systematically characterizing, analyzing, and mitigating the diverse threats facing MCP-based agent ecosystems. Existing security research remains fragmented across individual attack papers, isolated benchmarks, and point defense mechanisms. This paper presents MCPSHIELD, a comprehensive formal security framework for MCP-based AI agents. We make four principal contributions: (1) a hierarchical threat taxonomy comprising 7 threat categories and 23 distinct attack vectors organized across four attack surfaces, grounded in the analysis of 177,000+ MCP tools; (2) a formal verification model based on labeled transition systems with trust-boundary annotations that enables static and runtime analysis of MCP tool interaction chains; (3) a systematic comparative evaluation of 12 existing defense mechanisms, identifying coverage gaps across our threat taxonomy; and (4) a defense-in-depth reference architecture integrating capability-based access control, cryptographic tool attestation, information flow tracking, and runtime policy enforcement. Our analysis reveals that no existing single defense covers more than 34% of the identified threat landscape, whereas MCPSHIELD's integrated architecture achieves theoretical coverage of 91%. We further identify seven open research challenges that must be addressed to secure the next generation of agentic AI systems.**

***Index Terms*—Model Context Protocol, AI agent security, large language models, formal verification, threat modeling, tool poisoning, agentic AI**

I. Introduction

LARGE language model (LLM)-based AI agents represent a paradigm shift in software systems, evolving from stateless question-answering interfaces into autonomous entities capable of reasoning, planning, and executing multi-step workflows that modify real-world environments [1, 2]. Unlike traditional LLM applications, modern AI agents interact with external tools, databases, APIs, and other agents through structured protocols, creating complex trust relationships that extend far beyond the model's training boundary [3, 4].

Central to this transformation is the Model Context Protocol (MCP), an open standard introduced by Anthropic in November 2024 [5]. Built on JSON-RPC 2.0, MCP defines a bidirectional communication interface between LLM host applications (clients) and external capability providers (servers), standardizing how agents discover, invoke, and compose tools [6]. The protocol's adoption has been extraordinary: as of early 2026, the MCP ecosystem encompasses over 10,000 active servers, 177,000 registered tools, and 97 million monthly SDK downloads [7]. Major technology companies—including OpenAI, Google, Microsoft, and Amazon Web Services—have endorsed MCP through the Linux Foundation's Agentic AI Foundation (AAIF), establishing it as vendor-neutral infrastructure for the agentic AI era [8].

However, this rapid adoption has dramatically expanded the attack surface of AI agent systems. An empirical analysis of the MCP ecosystem reveals that "action" tools—those capable of directly modifying external environments—grew from 27% to 65% of all tools between November 2024 and February 2026 [7]. Each tool invocation creates a potential trust boundary violation, and the compositional nature of MCP interactions means that compromising a single server can cascade through multi-agent workflows. Recent work has demonstrated concrete attacks including tool poisoning (embedding malicious instructions in tool descriptions) [9, 10], rug pulls (mutating tool behavior post-approval) [11], cross-server data exfiltration [12], and privilege escalation through tool composition [13].

The security research community has responded with a growing but *fragmented* body of work. Benchmarks such as MCPTox [10], MCP-SafetyBench [14], and MCPSecBench [13] evaluate specific attack categories but use incompatible threat taxonomies. Defense proposals including ETDI [11], MCP-Guard [15], and MCPGuard [16] address individual attack vectors but lack integration. Enterprise security frameworks [17] provide high-level guidance without formal underpinnings. Most critically, no existing work provides a *unified formal model* that captures the security properties of MCP interactions in a verifiable manner.

This fragmentation poses three concrete problems. First, practitioners cannot systematically assess the security posture of an MCP deployment because no common threat vocabulary exists. Second, defense mechanisms cannot be meaningfully compared because they operate under different assumptions and threat models. Third, the absence of formal semantics for MCP security properties precludes automated verification and certified deployments.

基於 MCP 的 AI 代理正式安全框架：威脅分類、驗證模型與防禦機制

尼拉詹·阿查婭[1], 高拉夫·庫瑪·古普塔[2] [1]坎伯蘭大學
Email: nacharya39400@ucumberlands.edu [2]Youngstown State University
Email: guptagauravk1@gmail.com

arXiv:2604.05969v1 [cs.CR] 7 Apr 2026

摘要——Anthropic 於 2024 年 11 月推出的模型上下文協議（MCP），現由 Linux 基金會的代理 AI 基金會管理，已迅速成為連接基於大語言模型（LLM）的代理與外部工具及數據來源的事實標準，其 SDK 月下載量超過 9700 萬，註冊工具超過 177,000 種。然而，這種爆炸式採用也暴露了一個關鍵缺口：缺乏一個統一的、形式化的安全框架，能夠系統性地描述、分析和緩解 MCP 基於代理的生態系統所面臨的多樣化威脅。現有的安全研究分散於個別攻擊論文、孤立的基準測試和點防禦機制之中。本文提出 MCPSHIELD，一個針對 MCP 基於 AI 代理的全面形式化安全框架。我們做出四項主要貢獻：(1) 一個分層威脅分類法，包含 7 個威脅類別和 23 個獨特攻擊向量，這些向量分佈於四個攻擊表面，基於對 177,000+MCP 工具的分析；(2) 一個基於標記過渡系統的形式驗證模型，並帶有信任邊界註釋，該模型支持對 MCP 工具互動鏈進行靜態和時序分析；(3) 對 12 種現有防禦機制的系統性比較評估，識別出我們威脅分類法中的覆蓋缺口；以及 (4) 一個縱深防禦參考架構，整合基於能力的訪問控制、加密工具驗證、信息流追蹤和時序策略執行。我們的分析顯示，沒有任何現有的單一防禦機制能覆蓋超過 34% 的已識別威脅景觀，而 MCPSHIELD 的整合架構實現了 91% 的理論覆蓋率。我們進一步識別出七項必須解決的開放研究挑戰，以確保下一代代理 AI 系統的安全。

索引詞—模型上下文協定，AI 代理安全性，大型語言模型，形式驗證，威脅建模，工具投毒，代理式 AI

一、引言

ARGE 基於 L 語言模型（LLM）的 AI 代理代表了軟體系統範式的轉變，從無狀態的問答介面演變為具備推理、規劃及執行多步驟工作流以修改真實世界環境 [1, 2] 的自主實體。與傳統 LLM 應用不同，現代 AI 代理透過結構化協定與外部工具、資料庫、API 及其他代理進行互動，建立複雜的信任關係，其範圍遠超模型訓練邊界 [3, 4]。

本轉型的核心在於模型上下文協議（MCP），這是 Anthropic 於 11 月 2024 [5] 推出的開放標準。MCP 基於 JSON-RPC 2.0，定義了 LLM 主機應用程式與端（客戶端）與外部能力提供者（伺服器），標準化代理如何發現、調用及組裝工具 [6]。該協議的採用情況極為驚人：截至 2026 年初，MCP 生態系統包含超過 10,000 個活躍伺服器、177,000 個註冊工具，以及每月 9700 萬次的 SDK 下載 [7]。主要科技企業——包括 OpenAI、Google、Microsoft 和 Amazon Web Services——已透過 Linux Foundation 的代理 AI 基金會（AAIF）支持 MCP，將其確立為代理 AI 時代的中立基礎設施 [8]。

然而，這種快速採用大幅擴大了 AI 代理系統的攻擊面。對 MCP 生態系統的經驗分析顯示，「行動」工具（即能夠直接修改外部環境的工具）在 2024 年 11 月至 February 2026 [7] 期間，佔所有工具的比率從 27% 上升至 65%。每次工具調用都構成了潛在的信任邊界違反，而 MCP 互動的組合特性意味著，攻陷單一伺服器可能會導致多代理工作流的連鎖反應。近期研究已證明具體的攻擊案例，包括工具投毒（在工具描述中嵌入惡意指令）[9, 10]、、rug pulls（在獲批後篡改工具行為） [11]、、跨伺服器資料洩漏 [12]、，以及透過工具組合實現權限提升 [13]。

安全研究社群已回應，產出日益增長但零散的相關研究。例如 MCPTox [10]、 、MCP-SafetyBench [14]、與 MCPSecBench [13] 等基準測試雖評估特定攻擊類別，卻採用不相容的威脅分類法。防禦方案如 ETDI [11]、、MCP-Guard [15]、與 MCPGuard [16] 雖針對個別攻擊向量提出對策，卻缺乏整合性。企業安全架構 [17] 雖提供高層指導，卻無正式理論基礎。最關鍵的是，現存研究尚未提供統一的正式模型，以可驗證的方式捕捉 MCP 互動的安全屬性。

此碎片化現象帶來三個具體問題。首先，由於缺乏統一的威脅術語，從業者無法系統性地評估 MCP 部署的安全狀況。其次，由於各防禦機制基於不同的假設與威脅模型運作，因此無法進行有意義的比較。最後，MCP 安全屬性缺乏形式化語義，導致無法進行自動化驗證與認證部署。

This paper addresses these gaps through MCPSHIELD, a comprehensive formal security framework for MCP-based AI agents. Our contributions are:

- **Unified Threat Taxonomy (Section III).** We synthesize findings from 12 MCP security papers, 5 benchmarks, and the OWASP Top 10 for LLM Applications into a hierarchical taxonomy of 7 threat categories and 23 attack vectors organized across 4 attack surfaces, providing the first common vocabulary for MCP security.
- **Formal Verification Model (Section IV).** We introduce a labeled transition system with trust-boundary annotations ($\mathcal{M}_{\text{MCP}}$) that formalizes MCP interactions as state transitions. We define four fundamental security properties—*tool integrity*, *data confinement*, *privilege boundedness*, and *context isolation*—and provide decidability results for their verification.
- **Comparative Defense Analysis (Section V).** We systematically evaluate 12 existing defense mechanisms against our taxonomy, mapping coverage gaps and identifying architectural blind spots. We show that no individual defense covers more than 34% of the threat landscape.
- **Defense-in-Depth Reference Architecture (Section VI).** We propose MCPSHIELD, an integrated security architecture combining four complementary layers: capability-based access control, cryptographic tool attestation, information flow tracking, and runtime policy enforcement.

The remainder of this paper is organized as follows. Section II provides background on MCP architecture and related work. Section III presents our threat taxonomy. Section IV introduces the formal verification model. Section V analyzes existing defense mechanisms. Section VI describes the MCPSHIELD reference architecture. Section VII discusses implications and limitations. Section VIII identifies open research challenges. Section IX concludes.

II. Background and Related Work

A. The Model Context Protocol

MCP defines a client-server architecture where *hosts* (LLM applications) spawn *clients* that maintain stateful sessions with *servers* (external capability providers) [6]. The protocol operates over JSON-RPC 2.0 and supports three primary server capabilities:

- **Tools**: Executable functions that agents can invoke (e.g., file operations, API calls, database queries). Each tool is defined by a name, description, and JSON Schema input specification.
- **Resources**: Read-only data sources exposed via URI-based addressing (e.g., file contents, database records).
- **Prompts**: Reusable prompt templates that guide agent behavior for specific tasks.

The protocol lifecycle consists of four phases: *initialization* (capability negotiation via `initialize` handshake), *discovery* (tool/resource enumeration via `tools/list`), *operation* (tool invocation via `tools/call`), and *termination* (session closure) [18]. Servers may additionally request *sampling* from the host LLM and *elicitation* of user input, creating bidirectional information flows.

B. MCP Ecosystem Scale

Stein [7] conducted the first large-scale empirical study of the MCP ecosystem, monitoring 177,436 tools across public MCP server repositories between November 2024 and February 2026. Key findings relevant to security include:

- Software development accounts for 67% of all agent tools and 90% of MCP server downloads, concentrating risk in code-execution-adjacent operations.
- The proportion of "action" tools (those that modify external environments) rose from 27% to 65%, indicating a rapid shift from read-only to write-capable agent interactions.
- The long-tail distribution of server popularity creates a dual risk: heavily-used servers are high-value targets, while niche servers receive minimal security scrutiny.

C. Agent Communication Protocols Landscape

MCP is not the only agent communication protocol. Ehtesham et al. [19] survey four emerging protocols: MCP (tool invocation via JSON-RPC), Google's Agent-to-Agent Protocol (A2A, peer-to-peer task delegation) [20], the Agent Communication Protocol (ACP, RESTful multimodal messaging), and the Agent Network Protocol (ANP, decentralized agent discovery via W3C DIDs). Each protocol occupies a distinct niche—MCP for tool access, A2A for inter-agent collaboration—but their interoperability introduces additional security surfaces. An agent bridging MCP and A2A, for instance, must reconcile two distinct trust models, creating potential semantic gaps that adversaries can exploit.

D. Related Security Work

We organize related work into four categories.

MCP-Specific Security Research. Hou et al. [18] provided the first systematic study of MCP security, identifying 16 threat scenarios across 4 attacker types (malicious developers, external attackers, malicious users, and inherent security flaws) mapped to the MCP server lifecycle. Narajala and Habler [17] proposed enterprise mitigation strategies. Jamshidi et al. [21] analyzed tool poisoning, shadowing, and rug pull attacks, proposing RSA-based manifest signing and LLM-on-LLM semantic vetting. Jamshidi et al. [22] extended this with a secure tool manifest and digital signing framework.

MCP Benchmarks. MCPTox [10] benchmarks tool poisoning attacks across 45 real-world MCP servers with 353 tools and 1,312 malicious test cases spanning 11 risk categories. MCP-SafetyBench [14] provides a unified taxonomy of 20 attack types evaluated across 5 domains. MCPSecBench [13] formalizes a secure MCP specification and identifies 17 attack types across 4 attack surfaces. Despite overlapping scope, these benchmarks use incompatible taxonomies, making cross-comparison difficult.

MCP Defense Mechanisms. MCP-Guard [15] proposes a three-stage defense (static scanning, neural detection, LLM arbitration) achieving 89.63% accuracy. MCPGuard [16] focuses on automated vulnerability detection for MCP servers. ETDI [11] introduces OAuth-enhanced tool definitions with cryptographic identity verification. MCPS [23] implements ECDSA P-256 message signing with progressive trust levels.

本文通過 MCPSHIELD 這一針對基於 MCP 的 AI 代理的綜合形式安全框架，來彌補這些缺口。我們的貢獻如下：

- **統一威脅分類學（第三節）。我們綜合**
將來自 12 篇 MCP 安全論文、5 個基準測試以及 OWASP Top 10 for LLM Applications 的發現，整合為一個包含 7 個威脅類別和 23 個攻擊向量的層級分類體系，該體系按 4 個攻擊表面進行組織，為 MCP 安全提供首個通用術語庫。
- 形式驗證模型（第四節）。我們引入一個帶有信任邊界註釋的標記過渡系統（$\mathcal{M}$MCP），將 MCP 互動形式化為狀態遷移。我們定義了四個基本安全屬性——工具完整性、資料封閉性、權限有界性與情境隔離，並提供了其驗證的可判定性結果。
- 防禦機制比較分析（第五節）。我們系統性地評估 12 種現有防禦機制，並將其與我們的分類法進行比對，以找出覆蓋缺口並識別架構盲點。我們顯示，沒有任何單一防禦機制能涵蓋超過 34% 的威脅景況。
- 縱深防禦參考架構（第六節）。我們提出 MCPSHIELD，這是一種整合的安全架構，結合四個互補層級：基於能力的存取控制、加密工具驗證、資訊流程追蹤，以及執行時策略強制執行。

本文的其餘部分組織如下。第二部分提供 MCP 架構的背景與相關研究。第三部分提出我們的威脅分類法。第四部分介紹形式驗證模型。第五部分分析現有的防禦機制。第六部分描述 MCP-SHIELD 參考架構。第七部分討論影響與限制。第八部分識別開放的研究挑戰。第九部分總結。

二。背景與相關研究

A. 模型上下文協議

MCP 定義了一種客戶端 - 伺服器架構，其中主機（LLM 應用程式）會啟動客戶端，以與伺服器（外部能力提供者） [6] 維持有狀態的會話。該協議基於 JSON-RPC 2.0 運作，並支援三種主要伺服器
功能：

- 工具：代理可調用的可執行函數（例如，文件操作、API 調用、數據庫查詢）。每個工具都定義為名稱、描述和 JSON Schema 輸入規範。
- 資源：透過 URI 基址揭露的只讀資料來源（例如，檔案內容、資料庫記錄）。
- 提示詞：用於指導代理執行特定任務的可重複提示詞模板。

The protocollifecycle 包含四個階段：初始化（透過 initialize handshake 進行能力協商）、發現（透過 tools/list 進行工具/資源列舉）、執行（透過 tools/call 呼叫工具）以及終止（會話關閉） [18]。伺服器還可額外請求從主 LLM 進行採樣並誘導使用者輸入，從而建立雙向資訊流。

B. MCP 生態系規模

Stein [7] 進行了 MCP 生態系統的首次大規模實證研究，監控了 2024 年 11 月至 2 月期間公共 MCP 伺服器儲存庫中的 177,436 個工具。
ary 2026。與安全相關的主要發現如下：

- 軟體開發佔所有代理工具的 67% 及 MCP 伺服器下載的 90%，使風險集中在與程式碼執行相關的作業中。
- 「行動」工具（那些會修改外部環境）從 27% 上升至 65%，顯示從只讀到可寫代理互動的快速轉變。
- 伺服器人氣的長尾分佈造成雙重風險：使用頻繁的伺服器是高價值目標，而利基伺服器則幾乎不受安全審查。

C. 代理通訊協定概況

MCP 並非唯一的代理通訊協議。Ehte-sham 等人 [19] 調查了四種新興協議：MCP （透過 JSON-RPC 進行工具調用）、Google 的 Agent-to-Agent 協議（A2A，對等任務委託） [20]，代理通訊協議（ACP，基於 RESTful 的多模態訊息傳輸），以及代理網路協議（ANP，透過 W3C DIDs 進行去中心化代理發現）。每種協議佔據不同的生態位——MCP 用於工具存取，A2A 用於代理間協作——但它們的互操作性引入了額外的安全風險。例如，一個同時連接 MCP 與 A2A 的代理必須調和兩種不同的信任模型，這可能產生潛在的語義差距，讓攻擊者得以利用。

D. 相關安全工作

我們將相關工作分為四類。

MCP 特定安全研究。Hou 等人 [18] 提供了 MCP 安全的首个系统性研究，识别出 16 个威胁场景，涵盖 4 类攻击者（恶意开发者、外部攻击者、恶意用户和固有安全缺陷），并将其映射到 MCP 服务器生命周期。Narajala 和 Habler[17] 提出了企业缓解策略。Jamshidi 等人[21] 分析了工具投毒、影子化和 rug pull 攻击，提出了基于 RSA 的清单签名和 LLM-on-LLM 语义审查方案。Jamshidi 等人 [22] 在此基础上扩展了安全工具清单和数字签名框架。

MCP 基準測試。MCPTox [10] 基準測試工具中毒 - 針對 45 個真實世界 MCP 伺服器、353 個工具以及涵蓋 11 個風險類別的 1,312 個惡意測試案例進行攻擊測試。MCP-SafetyBench [14] 提供了一種統一的 20 種攻擊類型分類，並涵蓋 5 個領域進行評估。MCPSecBench [13]
正式規範了安全的 MCP 規範，並識別出涵蓋 4 個攻擊面的 17 種攻擊類型。儘管範圍有所重疊，但這些基準使用了不相容的分類法，導致跨基準比較變得困難。

MCP 防禦機制。MCP-Guard [15] 提出三階段防禦（靜態掃描、神經檢測、LLM 仲裁），準確率達 89.63%。MCPGuard [16] 專注於 MCP 伺服器的自動化漏洞檢測。ETDI [11] 引入強化 OAuth 的工具定義，並
密碼學的驗證
實現具有漸進式信任層級的 ECDSA P-256 訊息簽名。

General AI Agent Security. Deng et al. [4] identify four knowledge gaps in AI agent security: unpredictability of multi-step inputs, complexity in internal executions, variability of operational environments, and interactions with untrusted external entities. He et al. [24] report that none of 16 tested LLM agents achieved an overall safety score above 60%. Tang et al. [25] provide a comprehensive taxonomy of attacks and defenses for LLM-based agents. Dehghantanha and Homayoun [26] systematize the attack surface of agentic AI across tools, RAG, and multi-agent loops. The OWASP Top 10 for LLM Applications [27] identifies Excessive Agency (LLM06) and Supply Chain (LLM03) as critical risks directly relevant to MCP.

Differentiation. Unlike prior work, this paper provides: (a) a unified threat taxonomy reconciling all existing MCP-specific taxonomies, (b) the first formal model with verifiable security properties for MCP interactions, and (c) a defense-in-depth architecture informed by systematic coverage analysis of existing mechanisms.

III. Threat Taxonomy for MCP-Based Agents

We construct our threat taxonomy through a systematic process. First, we collected all attack vectors documented in 12 MCP security papers [9–18, 21, 22] and 4 general agent security surveys [4, 24–26]. We then performed open coding to identify common themes, merged overlapping categories, and organized the result into a hierarchical structure. The taxonomy was validated against the OWASP Top 10 for LLM Applications [27], the STRIDE framework [28], and the 177,000-tool empirical dataset [7].

A. Attack Surface Model

We identify four attack surfaces in MCP-based agent systems:

Definition 1 (MCP Attack Surfaces)**.** *An MCP agent system exposes four attack surfaces:*

1) $\mathcal{S}_{tool}$***: Tool Interface Surface.*** *The boundary between the LLM agent and MCP tool definitions, including tool descriptions, parameter schemas, and return values.*
2) $\mathcal{S}_{transport}$***: Transport Surface.*** *The communication channel between MCP clients and servers, including JSON-RPC messages, session state, and transport-layer security.*
3) $\mathcal{S}_{server}$***: Server Surface.*** *The MCP server implementation itself, including its authentication mechanisms, resource access patterns, and supply chain dependencies.*
4) $\mathcal{S}_{compose}$***: Composition Surface.*** *The emergent surface created when multiple MCP servers, tools, and agents interact within a single workflow or across protocol boundaries (MCP/A2A).*

B. Threat Categories

We organize 23 attack vectors into 7 threat categories mapped to these surfaces (Table I).

1) TC1: Tool Poisoning: Tool poisoning exploits the fundamental design of MCP, where tool descriptions are consumed by LLMs as natural language and thus can contain adversarial instructions [9]. Wang et al. [10] demonstrated that even highly capable models (o1-mini: 72.8% attack success rate) are vulnerable because the attack exploits their superior instruction-following abilities. We identify four variants:

TV1: Description Injection. Malicious instructions embedded in tool description fields are invisible to end users but parsed by the LLM, causing it to execute unintended actions. Invariant Labs demonstrated that a "Fact of the Day" tool could exfiltrate an entire WhatsApp chat history through this vector [9].

TV2: Schema Manipulation. Tool parameter schemas are modified to accept additional hidden fields that trigger side effects not visible in the tool's advertised interface.

TV3: Return Value Poisoning. Tool execution returns contain embedded adversarial instructions that hijack the agent's subsequent reasoning, chaining into further malicious tool invocations.

TV4: Tool Shadowing. A malicious server registers a tool with the same name or similar description as a legitimate tool, intercepting invocations intended for the legitimate version [21].

2) TC2: Rug Pull and Mutation Attacks: Rug pull attacks exploit MCP's dynamic nature, where tool definitions can change after initial user approval [11].

TV5: Post-Approval Mutation. A tool that initially behaves benignly modifies its description, parameters, or behavior after the user grants permission, effectively converting approved access into unauthorized capabilities.

TV6: Version Rollback. An MCP server rolls back to a previous tool version with known vulnerabilities or different behavior, circumventing security checks performed against the newer version.

TV7: Capability Escalation. A server gradually expands its tool's capabilities across sessions, each increment appearing minor but cumulatively acquiring powerful unauthorized access.

3) TC3: Cross-Server Data Leakage: When agents interact with multiple MCP servers in a single session, data from one server's context can leak to another [12].

TV8: Exfiltration via Logging. A malicious tool coerces the agent into invoking a logging or analytics tool to transmit sensitive data to an attacker-controlled endpoint. The Log-to-Leak attack [12] systematizes this as four injection components: Trigger, Tool Binding, Justification, and Pressure.

TV9: Context Bleed. Information from one MCP server's responses persists in the LLM's context window and is inadvertently included in requests to another server, violating data isolation expectations.

TV10: Channel Coercion. A malicious tool manipulates the agent into using an insecure communication channel (e.g., HTTP instead of HTTPS, unencrypted stdio) for data transfer.

TV11: Sampling Abuse. MCP servers can request LLM sampling from the host. A malicious server exploits this bidirectional channel to extract information from the host's context or influence the model's behavior for other servers.

通用 AI 代理安全。Deng 等人 [4] 指出 AI 代理安全存在四個知識缺口：多步驟輸入的不可預測性、內部執行的複雜性、操作環境的變異性，以及與不可信外部實體的互動。He 等人 [24] 報告稱，16 個測試過的 LLM 代理中，沒有任何一個的整體安全评分超過 60%。Tang 等人 [25] 為基於 LLM 的代理提供了攻擊與防禦的完整分類法。Dehghantanha 和 Homayoun[26] 系統化了代理式 AI 在工具、RAG 和多代理迴路中的攻擊面。LLM 應用 OWASP Top 10 [27] 將過度代理（LLM06）與供應鏈（LLM03）識別為與 MCP 直接相關的重大風險。

差異化。與先前研究不同，本文提供：(a) 統一威脅分類法，整合所有現有的 MCP 特定分類法；(b) 第一個具備可驗證安全屬性的 MCP 互動形式化模型；以及 (c) 基於現有機制系統性覆蓋分析所啟發的纵深防禦架構。

III. MCP 基體智能體的威脅分類

我們透過系統化流程建構威脅分類法。首先，我們收錄了 12 篇 MCP 安全論文 [9–18, 21, 22] 與 4 份通用代理安全調查 [4, 24–26] 中記錄的所有攻擊向量。接著，我們進行開放式編碼以識別共同主題，合併重疊類別，並將結果組織成層級結構。該分類法已針對 LLM 應用 OWASP Top 10 [27]、STRIDE 框架 [28], 以及包含 177,000 個工具的實證數據集 [7] 進行驗證。

A. 攻擊面模型

我們識別出基於 MCP 的代理系統中的四個攻擊面：

定義 *1*（*MCP* 攻擊面）。*MCP* 代理系統暴露四個攻擊面：

1) $\mathcal{S}_{tool}$*:* 工具介面介面。*LLM* 代理與 *MCP* 工具定義之間的邊界，包含工具描述、參數模式及回傳值。

2) $\mathcal{S}_{transport}$*:* 傳輸層。*MCP* 客戶端與伺服器之間的通訊通道，包含 *JSON-RPC* 訊息、會話狀態及傳輸層安全性。

3) $\mathcal{S}_{server}$*:* 伺服器表面。*MCPserver* 實作本身，包括其身份驗證機制、資源存取模式以及供應鏈依賴。

4) $\mathcal{S}_{compose}$*:* 組成表面。當多個 *MCP* 伺服器、工具與代理在單一工作流中互動，或跨越協議邊界（*MCP/A2A*）時所產生的突現表面。

B. 威脅類別

我們將 23 個攻擊向量分為 7 個威脅類別，並映射至這些表面（表 I）。

1) TC1：工具投毒：工具投毒利用了 MCP 的基本設計，其中工具描述被 LLM 作為自然語言消耗，因此可能包含敵對指令 [9]。Wang 等人 [10] 證明，即使高度具備能力的模型（o1-mini：72.8% 攻擊成功率）之所以易受攻擊，是因為攻擊利用了它們優異的指令遵循能力。我們識別出四種變體：

TV1:描述注入。惡意指令嵌入在工具描述欄位中，對終端用戶而言是無形的，但
經大型語言模型解析，導致其執行非預期行為。
Invariant Labs 證明，「每日事實」工具可透過此向量洩漏整個 WhatsApp 聊天記錄 [9]。

TV2：模式操作。工具參數模式是
修改以接受觸發側面效果的額外隱藏欄位
工具宣稱介面中無法見到的效果。
TV3：返回值中毒。工具執行返回 *con-*
包含嵌入的對抗性指令，劫持代理的
後續推理，鏈接至更多惡意工具調用。

TV4：工具影子。惡意伺服器註冊一個與合法工具同名或描述相似的工具，攔截針對合法版本 [21] 的呼叫。

2) TC2：抽資與突變攻擊：抽資攻擊
利用 MCP 的動態特性，工具定義可在初審用戶同意 [11] 後變更。

TV5：事後批准變異。一個最初行爲
benignly 修改其描述、參數或行為後
使用者授權後，實際上將已核准的存取權轉化為未經授權的能力。

TV6：版本回滾。MCP 伺服器回滾至具有已知漏洞或不同版本的先前工具
行為，繞過針對新版本執行的安全檢查。

TV7：能力升級。伺服器逐步在會話中擴展其工具的功能，每次增量看似微小，但累積起來卻獲得強大的未經授權能力。

cess.
3) TC3：跨伺服器資料洩漏：當代理互動時
在同一個會話中有多個 MCP 伺服器時，來自一個伺服器上下文的資料可能會洩漏到另一個伺服器 [12]。
TV8：透過日誌外洩。惡意工具強迫代理調用日誌或分析工具，將敏感資料傳輸至攻擊者控制的終端點。
Log-to-Leak 攻擊 [12] 將此過程系統化為四個注入組件：觸發、工具綁定、合理化與施壓。

TV9：情境洩漏。來自一個 MCP 伺服器回應的資訊會持續保留在 LLM 的上下文視窗中，且不適
錯誤地包含在對其他伺服器的請求中，違反了資料隔離的預期。

TV10：通道強制。惡意工具會誘導代理使用不安全的通訊通道（例如使用 HTTP 而非 HTTPS，或未加密的 stdio）進行資料傳輸。

TV11：採樣濫用。MCP 伺服器可請求主機進行 LLM 採樣。惡意伺服器利用此雙向通道，從主機的上下文中提取資訊，或影響模型行為以服務其他伺服器。

TABLE I
UNIFIED THREAT TAXONOMY FOR MCP-BASED AI AGENTS. ATTACK VECTORS ARE MAPPED TO ATTACK SURFACES ($\mathcal{S}$), ATTACKER TYPE, AND EXISTING BENCHMARK/PAPER COVERAGE (✓= COVERED, ○ = PARTIALLY COVERED, — = NOT COVERED).

Threat Category	Attack Vector	Surface	Attacker	MCPTox	SafetyBench	SecBench
TC1: Tool Poisoning	TV1: Description injection	$\mathcal{S}_{\text{tool}}$	Developer	✓	✓	✓
	TV2: Schema manipulation	$\mathcal{S}_{\text{tool}}$	Developer	○	—	✓
	TV3: Return value poisoning	$\mathcal{S}_{\text{tool}}$	External	✓	○	○
	TV4: Tool shadowing	$\mathcal{S}_{\text{tool}}$	Developer	—	—	✓
TC2: Rug Pull & Mutation	TV5: Post-approval mutation	$\mathcal{S}_{\text{server}}$	Developer	—	—	○
	TV6: Version rollback	$\mathcal{S}_{\text{server}}$	Developer	—	—	—
	TV7: Capability escalation	$\mathcal{S}_{\text{server}}$	Developer	—	—	—
TC3: Cross-Server Data Leakage	TV8: Exfiltration via logging	$\mathcal{S}_{\text{compose}}$	External	—	✓	○
	TV9: Context bleed	$\mathcal{S}_{\text{compose}}$	External	—	○	—
	TV10: Channel coercion	$\mathcal{S}_{\text{compose}}$	External	—	—	—
	TV11: Sampling abuse	$\mathcal{S}_{\text{transport}}$	Server	—	—	—
TC4: Privilege Escalation	TV12: Capability chaining	$\mathcal{S}_{\text{compose}}$	External	—	○	○
	TV13: Consent bypass	$\mathcal{S}_{\text{tool}}$	External	○	✓	✓
	TV14: Role confusion	$\mathcal{S}_{\text{transport}}$	External	—	—	—
TC5: Server Trust Violations	TV15: Impersonation	$\mathcal{S}_{\text{server}}$	External	—	—	✓
	TV16: Supply chain compromise	$\mathcal{S}_{\text{server}}$	Developer	—	—	—
	TV17: Dependency hijacking	$\mathcal{S}_{\text{server}}$	External	—	—	—
TC6: Context Manipulation	TV18: Prompt injection via tool	$\mathcal{S}_{\text{tool}}$	External	✓	✓	✓
	TV19: Memory poisoning	$\mathcal{S}_{\text{compose}}$	External	—	○	—
	TV20: Resource injection	$\mathcal{S}_{\text{tool}}$	External	○	○	○
TC7: Protocol-Level Vulnerabilities	TV21: Session hijacking	$\mathcal{S}_{\text{transport}}$	External	—	—	—
	TV22: Replay attacks	$\mathcal{S}_{\text{transport}}$	External	—	—	—
	TV23: Cross-protocol confusion	$\mathcal{S}_{\text{compose}}$	External	—	—	—

4) TC4: Privilege Escalation: Agents operating under the principle of least privilege may still be manipulated into exceeding their authorized capabilities [27].

TV12: Capability Chaining. An attacker composes individually benign tool invocations to achieve an unauthorized outcome. For example, a read-file tool combined with a send-email tool enables data exfiltration, though neither tool alone is malicious.

TV13: Consent Bypass. Adversarial tool descriptions manipulate the LLM into auto-approving actions that should require explicit user consent, exploiting the agent's tendency to follow instructions embedded in tool outputs.

TV14: Role Confusion. In multi-role agent architectures, a tool invocation blurs the boundary between user-level and system-level operations, causing the agent to execute privileged operations under a standard user context.

5) TC5: Server Trust Violations: The MCP ecosystem's open nature—anyone can publish a server—creates server-level trust challenges analogous to software supply chain attacks [29].

TV15: Impersonation. An attacker deploys an MCP server that mimics the identity (name, description, tool signatures) of a trusted server to intercept agent interactions.

TV16: Supply Chain Compromise. A legitimate MCP server's dependencies are compromised, introducing malicious behavior without modifying the server's own code.

TV17: Dependency Hijacking. A server's package name or repository is claimed by an attacker after the original maintainer abandons it, similar to npm/PyPI namespace hijacking.

6) TC6: Context Manipulation: These attacks target the LLM's reasoning process through its context window [30].

TV18: Prompt Injection via Tool. The canonical MCP attack: malicious content in tool descriptions or return values that hijacks the agent's behavior. MCPTox [10] categorizes 11 risk categories spanning information theft, harmful content generation, and system manipulation.

TV19: Memory Poisoning. In agents with persistent memory (e.g., conversation history, knowledge bases), adversarial tool outputs inject false information that corrupts future reasoning sessions.

TV20: Resource Injection. MCP resources (read-only data sources) are poisoned with adversarial content that influences the agent's tool selection and invocation patterns.

7) TC7: Protocol-Level Vulnerabilities: These attacks exploit weaknesses in MCP's transport and session management [31].

TV21: Session Hijacking. An attacker captures or predicts MCP session identifiers to inject commands into an active agent-server session.

TV22: Replay Attacks. Previously captured JSON-RPC messages are re-transmitted to trigger repeated tool invocations or re-establish expired sessions.

表 I
MCP 基礎 AI 代理的統一威脅分類法。攻擊向量已映射至攻擊表面 ($\mathcal{S}$)、攻擊者類型，以及現有基準/論文覆蓋範圍 (✓= 已覆蓋， ○ = 部分覆蓋，— = 未覆蓋)。

威脅類別	攻擊向量	表面	攻擊者	MCPTox	安全評估儀	SecBench
TC1：工具中毒	TV1：描述注入	$\mathcal{S}_{\text{tool}}$	開發者	✓	✓	✓
	TV2：模式操作	$\mathcal{S}_{\text{tool}}$	開發者	○	—	✓
	TV3：返回值中毒	$\mathcal{S}_{\text{tool}}$	外部	✓	○	○
	TV4：工具陰影遮蔽	$\mathcal{S}_{\text{tool}}$	開發者	—	—	✓
TC2：拉錢與突變	TV5：批准後突變	$\mathcal{S}_{\text{server}}$	開發者	—	—	○
	TV6：版本回滾	$\mathcal{S}_{\text{server}}$	開發者	—	—	—
	TV7：能力升級	$\mathcal{S}_{\text{server}}$	開發者	—	—	—
TC3：跨伺服器資料洩漏	TV8：透過日誌外洩	$\mathcal{S}_{\text{compose}}$	外部	—	✓	○
	TV9：情境洩漏	$\mathcal{S}$	外部	—	○	—
	TV10：頻道強制	$\mathcal{S}_{\text{compose}}$ 編寫	外部	—	—	—
	TV11：濫用抽樣	$\mathcal{S}_{\text{transport}}$	伺服器	—	—	—
TC4：權限升級	TV12：能力鏈式調用	$\mathcal{S}_{\text{compose}}$	外部	—	○	○
	TV13：同意跳過	$\mathcal{S}_{\text{tool}}$	外部	○	✓	✓
	TV14：角色混淆	$\mathcal{S}_{\text{transport}}$	外部	—	—	—
TC5：伺服器信任違規	TV15：假冒	$\mathcal{S}_{\text{server}}$	外部	—	—	✓
	TV16：供應鏈腐敗 mise	$\mathcal{S}_{\text{server}}$	開發者	—	—	—
	TV17：依賴劫持	$\mathcal{S}_{\text{server}}$	外部	—	—	—
TC6: 情境操縱	TV18：透過提示注入 tool	$\mathcal{S}_{\text{tool}}$	外部	✓	✓	✓
	TV19：記憶體中毒	$\mathcal{S}_{\text{compose}}$	外部	—	○	—
	TV20：資源注入	$\mathcal{S}_{\text{tool}}$	外部	○	○	○
TC7：協議層級漏洞	TV21：會話劫持	$\mathcal{S}_{\text{transport}}$	外部	—	—	—
	TV22：重放攻擊	$\mathcal{S}_{\text{transport}}$	外部	—	—	—
	TV23：跨協議混淆 - sion	$\mathcal{S}_{\text{compose}}$	外部	—	—	—

4) TC4：權限提升：在下的代理
最小權限原則仍可能被操縱以超出其授權能力 [27]。

TV12：能力鏈接。攻擊者組合個別無害的工具調用以達成未經授權的結果。例如，讀取檔案工具與發送郵件工具結合可導致資料洩漏，但單一工具本身並無惡意。

TV13：同意繞過。對抗性工具描述會操縱大型語言模型自動批准本應需要明確用戶同意的行動，利用該代理傾向於遵循嵌入在工具輸出中的指令。

TV14：角色混淆。在多角色代理架構中，工具調用模糊了用戶層級與系統層級操作之間的界線，導致代理在標準用戶上下文中執行特權操作。

5) TC5: 伺服器信任違規：*MCP* 生態系統的 opennature—任何人都可以發布伺服器——會產生類似軟體供應鏈攻擊的伺服器級信任挑戰 [29]。

TV15：假冒。攻擊者部署一個 MCP 伺服器，模擬可信伺服器的身份（名稱、描述、工具簽名），以攔截代理的互動。

TV16：供應鏈洩密。合法 MCP 伺服器之依賴項遭竄改，引入惡意行為，卻未修改伺服器自身程式碼。

TV17:DependencyHijacking。攻擊者聲稱接管了原始主機的伺服器套件名稱或儲存庫。
容器放棄它，類似於 npm/PyPI 命名空間劫持。
6) TC6：情境操控：這些攻擊透過其情境視窗 [30]針對 LLM 的推理過程進行攻擊。

TV18：透過工具進行的提示注入。標準的 MCP 攻擊：惡意內容出現在工具描述或返回值中，從而劫持代理的行為。MCPTox [10] 將風險分為 11 個類別，涵蓋資訊竊取、有害內容生成及系統操控。

TV19：記憶中毒。在具有持久記憶的代理（例如，對話歷史、知識庫）中，敵對工具輸出會注入錯誤資訊，從而腐蝕未來的推理會話。

TV20：資源注入。MCP 資源（只讀資料來源）被注入敵對內容，影響代理的工具選擇與呼叫模式。

7) TC7：協議層漏洞：這些攻擊利用 MCP 傳輸和會話管理 [31] 中的弱點。

TV21：會話竊取。攻擊者截獲或預測 MCP 會話識別碼，以向活躍的代理伺服器會話中注入指令。

TV22：重放攻擊。先前擷取的 JSON-RPC 訊息被重新傳送，以觸發重複的工具呼叫或重新建立已過期的會話。

TV23: Cross-Protocol Confusion. When an agent bridges MCP and another protocol (e.g., A2A), semantic differences in trust models, authentication mechanisms, or capability representations create exploitable gaps [19].

C. Taxonomy Coverage Analysis

Table I reveals significant gaps in existing benchmark coverage. Of 23 attack vectors, MCPTox covers 5 fully and 2 partially, MCP-SafetyBench covers 4 fully and 5 partially, and MCPSecBench covers 5 fully and 4 partially. No benchmark addresses rug pull mutations (TV5–TV7), protocol-level attacks (TV21–TV23), or supply chain threats (TV16–TV17). This confirms our thesis that existing work is fragmented and a unified framework is necessary.

IV. Formal Verification Model

We formalize MCP interactions as a labeled transition system with trust-boundary annotations. The goal is to define security properties precisely enough for automated verification.

A. System Model

Definition 2 (MCP Agent System)**.** *An MCP agent system is a tuple*

$$\mathcal{M} = (\mathcal{A}, \mathcal{S}, \mathcal{T}, \mathcal{R}, \mathcal{C}, \ell)$$

where:

- $\mathcal{A} = \{a_1, \ldots, a_n\}$ *is a finite set of agents (LLM instances);*
- $\mathcal{S} = \{s_1, \ldots, s_m\}$ *is a finite set of MCP servers;*
- $\mathcal{T} = \{t_1, \ldots, t_k\}$ *is a finite set of tools, where each* t_i *is hosted by some* $s_j \in \mathcal{S}$*;*
- $\mathcal{R} = \{r_1, \ldots, r_p\}$ *is a finite set of resources;*
- $\mathcal{C} \subseteq \mathcal{A} \times \mathcal{S}$ *is the set of active client connections;*
- $\ell \colon \mathcal{T} \cup \mathcal{R} \to \mathcal{L}$ *is a security labeling function mapping tools and resources to a security lattice* $\mathcal{L}$*.*

The security lattice $\mathcal{L}$ follows Denning [32], partially ordered by the *can-flow-to* relation $\sqsubseteq$, with $\bot$ (public) and $\top$ (restricted) as bounds.

Definition 3 (Trust Domain)**.** *A trust domain* $\mathcal{D}_i = (\mathcal{S}_i, \mathcal{T}_i, \pi_i)$ *groups a set of servers* $\mathcal{S}_i \subseteq \mathcal{S}$ *and their tools* $\mathcal{T}_i \subseteq \mathcal{T}$ *under a common trust policy* π_i*. Communication within a trust domain is considered trusted; cross-domain communication requires explicit verification.*

B. Labeled Transition System

Definition 4 (MCP Transition System)**.** *The MCP transition system is*

$$\mathcal{LTS} = (\Sigma, \Sigma_0, \Lambda, \to)$$

where:

- Σ *is the set of system states, where each state* $\sigma = (\mathcal{C}_\sigma, \mathcal{K}_\sigma, \mathcal{P}_\sigma)$ *consists of active connections* $\mathcal{C}_\sigma$*, knowledge state* $\mathcal{K}_\sigma$ *(information available to each agent), and permission state* $\mathcal{P}_\sigma$ *(granted capabilities);*
- $\Sigma_0 \subseteq \Sigma$ *is the set of initial states;*
- Λ *is the set of action labels, partitioned into:*
  - Λ_{disc}*: discovery (*`tools/list, resources/list`*)*
  - Λ_{inv}*: invocation (*`tools/call`*)*
  - Λ_{read}*: resource access (*`resources/read`*)*
  - Λ_{samp}*: sampling (*`sampling/createMessage`*)*
  - Λ_{admin}*: session mgmt. (*`initialize, shutdown`*)*
- $\to \subseteq \Sigma \times \Lambda \times \Sigma$ *is the transition relation.*

Each transition $\sigma \xrightarrow{\lambda} \sigma'$ represents a single MCP operation. A *trace* $\tau = \sigma_0 \xrightarrow{\lambda_1} \sigma_1 \xrightarrow{\lambda_2} \cdots \xrightarrow{\lambda_n} \sigma_n$ represents a complete interaction sequence. We write $\text{Traces}(\mathcal{LTS})$ for the set of all valid traces.

C. Security Properties

We define four fundamental security properties for MCP agent systems.

Property 1 (Tool Integrity)**.** *A system satisfies tool integrity if, for every trace* $\tau \in \mathit{Traces}(\mathcal{LTS})$ *and every tool invocation* $\sigma_i \xrightarrow{\texttt{call}(t)} \sigma_{i+1}$*, the tool definition* $\mathit{def}(t)$ *at time* i *is identical to the definition at the time of user approval:*

$$\forall \tau, \forall i : \sigma_i \xrightarrow{\texttt{call}(t)} \sigma_{i+1} \implies \mathit{def}_i(t) = \mathit{def}_{approved}(t) \quad (1)$$

This property defends against rug pull attacks (TC2) and tool shadowing (TV4).

Property 2 (Data Confinement)**.** *A system satisfies data confinement if information never flows from a higher security level to a lower one across trust domain boundaries. For any trace* τ *and any cross-domain data flow* $(d, \mathcal{D}_i, \mathcal{D}_j)$ *where* $i \neq j$*:*

$$\ell(d) \sqsubseteq \ell(\mathcal{D}_j) \quad \textit{or} \quad \mathit{declassify}(d, \mathcal{D}_j) \text{ is explicitly authorized} \quad (2)$$

This property defends against cross-server data leakage (TC3).

Property 3 (Privilege Boundedness)**.** *A system satisfies privilege boundedness if, for every trace* τ *and every state* σ_i*, the effective permissions of each agent* a *do not exceed the intersection of the agent's granted capabilities and the invoked tools' declared permissions:*

$$\forall \tau, \forall i, \forall a \in \mathcal{A} : \mathit{eff}(\sigma_i, a) \subseteq \mathit{cap}(a) \cap \bigcup_{t \in \text{active}(\sigma_i, a)} \mathit{perm}(t) \quad (3)$$

This property defends against privilege escalation (TC4), including capability chaining.

Property 4 (Context Isolation)**.** *A system satisfies context isolation if the knowledge state of an agent with respect to one server cannot influence its behavior toward another server without explicit cross-domain authorization:*

$$\forall \tau, \forall s_i, s_j \in \mathcal{S},\ i \neq j : \mathcal{K}_\sigma^{s_i}(a) \cap \mathit{input}\left(\sigma \xrightarrow{\texttt{call}(t_{s_j})}\right) = \emptyset \quad (4)$$

unless a cross-domain flow policy explicitly permits the transfer. This property defends against context bleed (TV9) and memory poisoning (TV19).

TV23：跨協議混淆。當代理橋接 MCP 與另一種協議（例如 A2A）的語義差異在信任模型、驗證機制或能力表示中會產生可被利用的漏洞 [19]。

C. 分類覆蓋率分析

表 I 顯示現有基準測試在覆蓋範圍上存在顯著缺口。在 23 個攻擊向量中，MCPTox 完全覆蓋 5 個、部分覆蓋 2 個；MCP-SafetyBench 完全覆蓋 4 個、部分覆蓋 5 個；MCPSecBench 完全覆蓋 5 個、部分覆蓋 4 個。沒有任何基準測試涵蓋騙錢變體（TV5–TV7）、協議層攻擊（TV21–TV23）或供應鏈威脅（TV16–TV17）。這證實了我們的假設：現有研究支離破碎，亟需建立統一的框架。

IV. 形式驗證模型

我們將 MCP 互動形式化為帶有信任邊界註釋的標記過渡系統。目標是精確定義安全屬性，以支援自動化驗證。

A. 系統模型

定義 *2*（*MCP* 代理系統）。*MCP* 代理系統是一個元組

$$\mathcal{M} = (\mathcal{A}, \mathcal{S}, \mathcal{T}, \mathcal{R}, \mathcal{C}, \ell)$$

其中：

- $\mathcal{A} = \{a_1, \ldots, a_n\}$ *is a finite set of agents (LLM instances);*
- $\mathcal{S} = \{s_1, \ldots, s_m\}$ 是一個有限的 *MCP* 伺服器集合；
- $\mathcal{T} = \{t_1, \ldots, t_k\}$ 是一個有限工具集合，其中每個 t_i 都由某個 $s_j \in \mathcal{S}$ 託管；
- $\mathcal{R} = \{r_1, \ldots, r_p\}$ 是一個有限資源集合；
- $\mathcal{C} \subseteq \mathcal{A} \times \mathcal{S}$ 爲活躍客戶端連線的集合；
- ℓ*:* $\mathcal{T} \cup \mathcal{R} \to \mathcal{L}$ 是一個將工具和資源映射到安全格 $\mathcal{L}$ 的安全標記函數。

安全格網 $\mathcal{L}$ 遵循 Denning [32],的部分或有序性，由 can-flow-to 關係 $\sqsubseteq$定義，其中 $\bot$ (公開)和 $\top$(受限)為邊界。

定義 *3*（信任領域）。信任領域 $\mathcal{D}_i = (\mathcal{S}_i, \mathcal{T}_i, \pi_i)$將一組伺服器 $\mathcal{S}_i \subseteq \mathcal{S}$ 及其工具 $\mathcal{T}_i \subseteq \mathcal{T}$歸於共同的信任策略 π_i之下。信任領域內的通信被視爲可信；跨領域通信需要明確驗證。

B. 標記過渡系統

定義 *4*（*MCP* 遷移系統）。*MCP* 遷移系統是

$$\mathcal{LTS} = (\Sigma, \Sigma_0, \Lambda, \to)$$

其中：

- Σ 爲系統狀態集合，其中每個狀態 $\sigma = (\mathcal{C}_\sigma, \mathcal{K}_\sigma, \mathcal{P}_\sigma)$ 包含活躍連線 $\mathcal{C}_\sigma$、知識狀態 $\mathcal{K}_\sigma$（每個代理可獲得的資訊），以及許可狀態 $\mathcal{P}_\sigma$（已授予的能力）；
- $\Sigma_0 \subseteq \Sigma$ 爲初始狀態集合；
- Λ是動作標籤的集合，分爲：
  - Λ_{disc}*:* 發現 *(*工具/清單，資源/清單*)*
  - Λ_{inv}：呼叫（工具/呼叫）
  - Λ_{read}：資源存取 *(*資源/讀取*)*
  - Λ_{samp}：採樣 *(*採樣/建立訊息*)*
  - Λ_{admin}*:* 會話管理 *(*初始化，關閉*)*
- $\to \subseteq \Sigma \times \Lambda \times \Sigma$ 是遷移關係。

每個過渡 $\sigma \xrightarrow{\lambda} \sigma'$ 代表單一 MCP 操作。一個追蹤 $\tau = \sigma_0 \xrightarrow{\lambda_1} \sigma_1 \xrightarrow{\lambda_2} \cdots \xrightarrow{\lambda_n} \sigma_n$ 代表完整的互動序列。我們用 $\text{Traces}(\mathcal{LTS})$ 表示所有有效追蹤的集合。

C. 安全屬性

我們為 MCP 代理系統定義了四項基本安全屬性。

屬性 *1*（工具完整性）。若系統滿足工具完整性，則對於每個跡象 $\tau \in \mathit{Traces}(\mathcal{LTS})$ 與每次工具調用 $\sigma_i \xrightarrow{\texttt{call}(t)} \sigma_{i+1}$，在時間 i 的工具定義 $\mathit{def}(t)$ 必須與用戶批准時的定義相同：

$$\forall \tau, \forall i : \sigma_i \xrightarrow{\texttt{call}(t)} \sigma_{i+1} \implies \mathit{def}_i(t) = \mathit{def}_{approved}(t) \quad (1)$$

此屬性可防禦地毯式抽逃攻擊（*TC2*）與工具遮蔽（*TV4*）。

屬性 *2*（資料封閉）。若資訊從未跨越信任領域邊界從較高安全等級流向較低安全等級，則該系統滿足資料封閉。對於任何軌跡 τ 及任何跨領域資料流 $(d, \mathcal{D}_i, \mathcal{D}_j)$，若 $i \neq j$：

$$\ell(d) \sqsubseteq \ell(\mathcal{D}_j) \quad \textit{or} \quad \mathit{declassify}(d, \mathcal{D}_j) \text{ 已明確授權 } (2)$$

此功能可防止跨伺服器資料洩漏（*TC3*）。

屬性 *3*（權限有界性）。若系統滿足權限有界性，則對於每個軌跡 τ 和每個狀態 σ_i，每個代理 a 的有效權限不得超過該代理所獲授權能與所調用工具所聲明權限的交集：

$$\forall \tau, \forall i, \forall a \in \mathcal{A} : \mathit{eff}(\sigma_i, a) \subseteq \mathit{cap}(a) \cap \bigcup_{t \in \text{active}(\sigma_i, a)} \mathit{perm}(t) \ (3)$$

此屬性 防範權限提升（*TC4*），包含能力鏈接。

屬性 *4*（環境隔離）。若一系統的知識狀態無法影響其對另一服務器的行爲，除非有明確的跨領域授權，則該系統滿足環境隔離。

$$\forall \tau, \forall s_i, s_j \in \mathcal{S},\ i \neq j : \mathcal{K}_\sigma^{s_i}(a) \cap \mathit{input}\left(\sigma \xrightarrow{\texttt{call}(t_{s_j})}\right) = \emptyset \quad (4)$$

除非跨域流量策略明確允許轉移。此屬性可防禦上下文洩漏 *(TV9)* 與記憶體投毒 *(TV19)*。

D. Verification Approach

Theorem 1 (Decidability of Tool Integrity)**.** *Tool integrity (Property 1) is decidable in $O(|\tau| \cdot |T|)$ time by maintaining a cryptographic hash of each tool's definition at approval time and comparing it before each invocation.*

Proof. At approval time, compute $h_t = H(\text{def}(t))$ for each tool t, where H is a collision-resistant hash function. Before each invocation $\texttt{call}(t)$, compute $h'_t = H(\text{def}_{\text{current}}(t))$. The check $h_t = h'_t$ requires $O(1)$ per invocation, and there are at most $|\tau|$ invocations across $|T|$ tools. □

Theorem 2 (Decidability of Data Confinement)**.** *Data confinement (Property 2) is decidable when the number of trust domains and security levels is finite, by constructing the product automaton of the MCP transition system and the information flow lattice [32].*

The verification of privilege boundedness and context isolation requires tracking capability sets and knowledge states through traces, which is decidable for finite-state systems but may require abstraction for practical MCP deployments.

We note that runtime enforcement of these properties can leverage security automata [33], where a monitor observes the trace of MCP operations and terminates (or modifies, following edit automata [34]) any execution that would violate a security property.

V. Comparative Analysis of Defense Mechanisms

We systematically evaluate 12 existing defense mechanisms against our threat taxonomy. For each mechanism, we assess: (a) which threat categories it addresses, (b) its enforcement model (preventive, detective, or reactive), (c) its deployment requirements, and (d) its performance overhead.

A. Defense Mechanism Catalog

D1: ETDI (Enhanced Tool Definition Interface) [11]. Extends MCP tool definitions with OAuth 2.0-based cryptographic identity verification, immutable versioned definitions, and policy-based access control. Addresses TC2 (rug pulls) through version pinning and TC4 (privilege escalation) through fine-grained permission policies. Does not address TC3, TC6, or TC7.

D2: MCP-Guard [15]. Three-stage defense: Stage 1 performs lightweight pattern-based static scanning (<2ms latency), Stage 2 uses E5 text embeddings for deep neural detection, and Stage 3 employs LLM arbitration for ambiguous cases. Achieves 89.63% accuracy and 89.07% F1-score. Primarily addresses TC1 (tool poisoning) and TC6 (prompt injection). Limited coverage of TC2, TC3, TC5.

D3: MCPGuard [16]. Automated vulnerability detection focusing on agent hijacking from protocol design deficiencies, traditional web vulnerabilities in MCP servers, and supply chain risks. Addresses TC1, partial TC5, but is detective-only (identifies vulnerabilities but does not enforce policies at runtime).

D4: Secure Tool Manifest [21, 22]. RSA-based manifest signing combined with LLM-on-LLM semantic vetting and heuristic guardrails. Addresses TC1 (tool poisoning), TC2 (rug pulls through signed manifests), and partial TC5 (server integrity through signing). Does not address TC3, TC4, or TC7.

D5: MCPS (MCP Secure) [23]. ECDSA P-256 cryptographic identity and message signing with progressive trust levels L0–L4. Every JSON-RPC message wrapped in a signed envelope with nonce-based replay protection. Addresses TC5 (server trust), TC7 (session hijacking, replay attacks). Does not address TC1, TC3, or TC6.

D6: MCP Specification Security Principles [6]. The MCP specification defines security principles around user consent, data privacy, tool safety, and LLM sampling controls, but enforcement is left to implementors. Provides partial TC4 coverage through consent requirements. Insufficient as a standalone defense.

D7–D8: Benchmark-Derived Defenses. MCP-SafetyBench [14] and MCPSecBench [13] include evaluation of model-level defenses (system prompts, safety training). MCPSecBench reports that current protection mechanisms achieve an average success rate below 30%, indicating fundamental insufficiency.

D9: Enterprise Security Frameworks [17]. Comprehensive mitigation strategies including threat modeling, access control policies, monitoring, and incident response. High-level architectural guidance without formal properties or automated enforcement.

D10: Microsoft Agent Governance Toolkit [35]. Runtime security with cryptographic identity (DIDs/Ed25519), dynamic execution rings inspired by CPU privilege levels, saga orchestration, and kill switch. Addresses TC4 (privilege escalation), TC5 (server trust), partial TC7. Designed for Microsoft Copilot ecosystem; MCP integration is indirect.

D11: OWASP Agentic AI Guidance [27]. Risk taxonomy and mitigation guidance for Excessive Agency, Supply Chain, and Prompt Injection risks. Addresses awareness but provides no enforceable mechanism.

D12: Zero Trust Architecture Principles [36]. NIST SP 800-207 principles applied to MCP: never trust implicitly, verify every tool invocation, enforce least privilege. Provides architectural guidance relevant to all threat categories but requires MCP-specific instantiation.

B. Coverage Gap Analysis

Table II maps each defense mechanism to the threat categories it covers.

Key findings from the coverage analysis:

1) **No single defense exceeds 34% coverage.** MCPSecBench's evaluation-based approach achieves the broadest coverage but remains partial across most categories.
2) **Cross-server data leakage (TC3) is the least defended category.** Only enterprise frameworks and zero trust principles provide partial coverage; no mechanism enforces data confinement formally.
3) **Protocol-level attacks (TC7) lack dedicated defenses.** Only MCPS addresses replay attacks directly; session hijacking and cross-protocol confusion remain unaddressed.

D. 驗證方法

定理 1（工具完整性可判定性）。工具完整性 *(Property 1)* 可在 $O(|\tau| \cdot |T|)$ 時間內透過於核准時維護每個工具的定義的加密雜湊值，並在每次呼叫前進行比對而判定。

證明。在批准時，對每個工具 t 計算 $h_t = H(\text{def}(t))$，其中 H 為抗碰撞雜湊函數。在每次呼叫 t 之前，計算 $h'_t = H(\text{def}_{\text{current}}(t))$。檢查 $h_t = h'_t$ 每次呼叫需要 $O(1)$，且 $|T|$ 個工具上的 $|\tau|$ 次呼叫總數最多。 □

定理 *2*（資料封閉的可判定性）。當信任領域與安全層級的數量爲有限時，資料封閉（性質 *2*）是可判定的，方法是構建 *MCP* 遷移系統與資訊流格 *[32]* 的產物自動機。

驗證權限有界與上下文隔離需要透過追蹤能力集合與知識狀態來實現，此過程在有限狀態系統中是可判別的，但在實際的 MCP 部署中可能需要抽象。

我們注意到，這些屬性的執行時強制可以利用安全自動機 [33]，，其中監控器觀察 MCP 操作的追蹤，並終止（或根據 edit 自動機 [34] 進行修改）任何違反安全屬性的執行。

V. 防禦機制之比較分析

我們系統性地評估 12 種現有防禦機制，並將其與我們的威脅分類法進行對照。針對每一種機制，我們評估：(a) 其所針對的威脅類別，(b) 其執行
模型（預防性、偵測性或反應性）、(c) 其部署需求，以及 (d) 其性能超載。

A. 防禦機制目錄

D1: ETDI (增強型工具定義介面) [11]。擴展 MCP 工具定義，加入基於 OAuth 2.0 的加密身份驗證、不可變版本化定義，以及基於策略的存取控制。透過版本鎖定處理 TC2（騙局），透過細粒度權限策略處理 TC4（權限提升）。未處理 TC3、TC6 或 TC7。

D2: MCP-Guard [15]。三階段防禦：第一階段執行輕量級基於模式的靜態掃描（<2ms 延遲），第二階段使用 E5 文字嵌入進行深度神經檢測，第三階段採用 LLM 仲裁處理模糊案例。準確率達 89.63%，F1 分數為 89.07%。主要針對 TC1（工具投毒）和 TC6（提示注入）進行防護。TC2、TC3、TC5 的覆蓋範圍有限。

D3: MCPGuard [16]。自動化漏洞檢測，專注於因協議設計缺陷導致的代理劫持、MCP 伺服器中的傳統 Web 漏洞，以及供應鏈風險。解決 TC1 及部分 TC5，但僅為偵測型（可識別漏洞，但無法在運行時強制執行策略）。

D4：安全工具清單 [21, 22]。**基於 RSA 的清單**簽署結合 LLM-on-LLM 語義審查與直覺性防護欄。解決 TC1（工具投毒）、TC2（透過簽署清單的騙局）以及部分 TC5（透過簽署確保伺服器完整性）。未解決 TC3、TC4 或 TC7。

D5: MCPS (MCP Secure) [23]。使用 ECDSA P-256 進行加密身份驗證與訊息簽署，並具備逐步信任等級 L0 至 L4。每個 JSON-RPC 訊息均包裹在帶有非重複數值（nonce）防重放保護的簽名信封中。涵蓋 TC5（伺服器信任）與 TC7（會話劫持、重放攻擊）。未處理 TC1、TC3 或 TC6。

D6: MCP 規範安全原則 [6]。MCP 規範定義了關於使用者同意、資料隱私、工具安全性及 LLM 採樣控制的的安全原則，但執行責任交由實作者承擔。透過同意要求提供部分 TC4 覆蓋。單獨作為防禦手段不足。

D7–D8：基於基準的防禦措施。MCP-SafetyBench [14] 與 MCP-SecBench [13]包含對模型層級防禦（系統提示、安全訓練）的評估。MCP-SecBench 報告顯示，當前的保護機制平均成功率低於 30%，表明存在根本性的不足。

D9: 企業安全架構 [17]。涵蓋全面的緩解策略，包括威脅建模、存取控制政策、監控與事件回應。提供高階架構指導，無正式屬性或自動化執行。

D10: Microsoft Agent Governance Toolkit [35]。具備加密身份（DIDs/Ed25519）的運行時安全性、受 CPU 權限等級啟發的動態執行環、Saga 編排以及殺手開。解決 TC4（權限提升）、TC5（伺服器信任）及部分 TC7。設計用於
Microsoft Copilot 生態系；MCP 整合為間接方式。

D11: OWASP 代理式 AI 指引 [27]。針對過度代理、供應鏈及提示注入風險的風險分類與緩解指引。提升意識，但無強制執行機制。

D12：零信任架構原則 [36]。將 NIST SP 800-207 原則應用於 MCP：永不默認信任，驗證每次工具調用，強制最小權限。提供適用於所有威脅類別的架構指導，但需要針對 MCP 進行具體實現。

B. 覆蓋缺口分析

表 II 將每項防禦機制映射至其所涵蓋的威脅類別。

覆蓋分析的主要發現：

1) **No 單一 防禦覆蓋率超過 34%。**
基於評估的方法雖能提供最廣泛的覆蓋範圍，但在大多數情況下仍顯不足。
MCPSecBench 的
分類。
2) **跨伺服器資料洩漏 (TC3) 是防禦最薄弱的類別。僅限企業框架與零信任原則**
請提供部分覆蓋；沒有機制能正式強制資料封閉。
。
3) **協議層攻擊（TC7）缺乏專用的防禦措施。**
僅 MCPS 地址能直接還原攻擊；會話高 -
jacking 與跨協議混淆問題仍未解決。

TABLE II
DEFENSE MECHANISM COVERAGE ACROSS THREAT CATEGORIES. ● = FULL COVERAGE, ◐ = PARTIAL COVERAGE, · = NO COVERAGE. MAX COVERAGE COLUMN SHOWS THE PERCENTAGE OF THE 23 ATTACK VECTORS ADDRESSED.

Defense	TC1	TC2	TC3	TC4	TC5	TC6	TC7	Cov.
D1: ETDI	·	●	·	◐	◐	·	·	22%
D2: MCP-Guard	●	·	·	·	·	●	·	30%
D3: MCPGuard	●	·	·	·	◐	·	·	22%
D4: Manifest	●	●	·	·	◐	·	·	26%
D5: MCPS	·	·	·	·	●	·	●	26%
D6: Spec	·	·	·	◐	·	·	·	9%
D7: SafetyB.	◐	·	◐	◐	·	◐	·	17%
D8: SecBench	◐	◐	◐	◐	◐	◐	·	34%
D9: Enterprise	◐	◐	◐	◐	◐	◐	◐	30%
D10: MS Gov.	·	·	·	●	●	·	◐	26%
D11: OWASP	◐	·	·	◐	◐	◐	·	17%
D12: Zero Trust	◐	◐	◐	◐	◐	◐	◐	30%

4) **Rug pull defenses (TC2) rely solely on versioning.** ETDI and secure manifests prevent post-approval mutation through version pinning, but capability escalation (TV7) across sessions is not addressed.
5) **Composition attacks ($\mathcal{S}_{\text{compose}}$) lack compositional defenses.** Individual tool-level defenses do not account for emergent risks from tool composition.

VI. MCPSHIELD: DEFENSE-IN-DEPTH ARCHITECTURE

Based on the coverage gap analysis, we propose MCPSHIELD, a layered security architecture that integrates four complementary defense layers to achieve comprehensive coverage across all seven threat categories.

A. Architectural Overview

MCPSHIELD follows a defense-in-depth strategy inspired by zero trust principles [36] and capability-based security [37]. The architecture consists of four layers:

1) **Layer 1: Capability-Based Access Control (L-CAC).** Governs what each agent can do by restricting tool invocations to explicitly granted capabilities.
2) **Layer 2: Cryptographic Tool Attestation (L-CTA).** Ensures what each tool *is* by verifying tool identity, integrity, and provenance at every invocation.
3) **Layer 3: Information Flow Tracking (L-IFT).** Controls where data *goes* by enforcing the data confinement property across trust domain boundaries.
4) **Layer 4: Runtime Policy Enforcement (L-RPE).** Monitors *how* interactions unfold by observing execution traces and enforcing security policies as security automata [33].

B. Layer 1: Capability-Based Access Control

Following Dennis and Van Horn [37] and the principle of least privilege, L-CAC models each agent's authority as a *capability set*—a collection of unforgeable tokens, each granting access to a specific tool with specific parameters.

Definition 5 (MCP Capability)**.** *An MCP capability is a tuple*

$$c = (t, params, scope, ttl, \sigma_c)$$

where $t \in \mathcal{T}$ identifies the tool, $params \subseteq Schema(t)$ restricts allowed parameters, $scope \in \{read, write, execute\}$ limits the operation type, ttl is a time-to-live, and σ_c is a cryptographic signature binding the capability to the issuing authority.

Before each tool invocation $\texttt{call}(t, \text{args})$, the L-CAC layer verifies:

1) The agent holds a valid capability c for tool t;
2) $\text{args} \subseteq \text{params}(c)$;
3) The operation is within $\text{scope}(c)$;
4) The capability has not expired (ttl not exceeded).

Composition Control. To defend against capability chaining (TV12), L-CAC introduces *composition policies* that restrict which tools can be invoked in sequence. A composition policy

$$\rho \colon \mathcal{T} \times \mathcal{T} \to \{\text{allow}, \text{deny}, \text{audit}\}$$

specifies whether tool t_j may be invoked after t_i within a single trace.

C. Layer 2: Cryptographic Tool Attestation

L-CTA ensures tool integrity (Property 1) by combining ideas from ETDI [11], secure manifests [22], and MCPS [23].

Each MCP server publishes a signed *tool attestation record*:

$$\text{TAR}(t) = \text{Sign}_{sk_s}\big(H(\text{def}(t)) \| \text{version}(t) \| \text{timestamp} \| \text{deps}(t)\big) \quad (5)$$

where sk_s is the server's private signing key, H is a collision-resistant hash function, and $\text{deps}(t)$ is a hash of the tool's dependency tree.

The L-CTA layer maintains an *attestation log* (analogous to certificate transparency) that records all tool attestation records. Before each invocation:

1) Verify $\text{TAR}(t)$ against the server's public key;
2) Compare $H(\text{def}_{\text{current}}(t))$ with the approved hash;
3) Verify $\text{version}(t)$ has not regressed (defending against TV6);
4) Check $\text{deps}(t)$ against known-good dependency hashes (defending against TV16–TV17).

表 II
威脅類別的防禦機制覆蓋範圍。● = 全面覆蓋，◐ = 部分覆蓋，· = 無覆蓋。最大覆蓋率欄位顯示已處理的 23 種攻擊向量的百分比。

防禦	TC1	TC2	TC3	TC4	TC5	TC6	TC7	Cov.
D1: ETDI	·	●	·	◐	◐	·	·	22%
D2: MCP 守護	●	·	·	·	·	●	·	30%
D3：MCPGuard	●	·	·	·	◐	·	·	22%
D4：Manifest	●	●	·	·	◐	·	·	26%
D5: MCPS	·	·	·	·	●	·	●	26%
D6: 規格	·	·	·	◐	·	·	·	9%
D7: 安全 B。	◐	·	◐	◐	·	◐	·	17%
D8: SecBench	◐	◐	◐	◐	◐	◐	·	34%
D9：企業	◐	◐	◐	◐	◐	◐	◐	30%
D10: 馬薩諸塞州政府。	·	·	·	●	●	·	◐	26%
D11: OWASP	◐	·	·	◐	◐	◐	·	17%
D12：零信任	◐	◐	◐	◐	◐	◐	◐	30%

4) 拉鉤防禦 (TC2) 僅依賴版本管理。ETDI 與安全清單透過版本鎖定防止審批後的變異，但跨會話的能力提升 (TV7) 尚未解決。
5) 組合攻擊 ($\mathcal{S}_{\text{compose}}$) 缺乏組合防禦。個別的工具層級防禦無法應對工具組合所產生的新興風險。

VI. MCPSHIELD：纵深防禦架構

基於覆蓋缺口分析，我們提出 MCP-SHIELD，一種分層安全架構，整合四個互補的防禦層，以實現對所有七類威脅的全面覆蓋。

A. 架構概覽

MCPSHIELD 採用基於零信任原則 [36] 與能力基礎安全 [37] 的纵深防禦策略。其架構包含四層：

1) 第一層：基於能力的訪問控制（L-CAC）。透過將工具調用限制於明確授予的能力，規範每個代理可執行的動作。

2) 第二層：加密工具驗證（L-CTA）。透過在每次調用時驗證工具的身份、完整性與來源，確保工具確實為預期之物。

3) 第 3 層：資訊流程追蹤 (L-IFT)。透過在信任領域邊界強制執行資料封閉屬性，來控制資料流向。

4) 第 4 層：執行時策略執行（L-RPE）。透過觀察執行追蹤來監控互動的展開，並作為安全自動機 [33] 強制執行安全策略。

B. 第一層：基於能力的存取控制

遵循 Dennis 和 Van Horn [37] 以及最小權限原則，L-CAC 將每個代理的權限建模為能力集合——即一組不可偽造的令牌，每個令牌授予對特定工具及其特定參數的存取權。

定義 *5*（*MCP* 能力）。*MCP* 能力是一個元組

$$c = (t, params, scope, ttl, \sigma_c)$$

其中 $t \in \mathcal{T}$ 識別工具，參數 $\subseteq Schema(t)$ 限制允許的參數，範圍 $\in \{read, write, execute\}$ 限制操作類型，ttl 爲生存時間，且 σ_c 是將能力綁定至發證機構的加密簽名。

在每次工具調用 call(t, args) 之前，L-CAC 層會驗證：

1) 該代理持有有效能力 c 用於工具 t；
2) $\text{args} \subseteq \text{params}(c)$;
3) 此操作在範圍內 (c)；
4) 能力尚未過期（ttl 未超過）。

組件控制。為防禦能力鏈接（TV12），L-CAC 引入組件策略，以限制可被順序調用的工具。組件策略

$$\rho \colon \mathcal{T} \times \mathcal{T} \to \{\text{allow}, \text{deny}, \text{audit}\}$$

指定工具 t_j 是否可在單一追蹤中於 t_i 之後被調用。

C. 第二層：密碼學工具驗證

L-CTA 透過整合來自 ETDI [11], 安全清單 [22], 與 MCPS [23] 的概念，確保工具完整性（屬性 1）。每個 MCP 伺服器會發布指定的工具驗證記錄：

$$\text{TAR}(t) = \text{Sign}_{sk_s}\big(H(\text{def}(t)) \| \text{version}(t) \| \text{timestamp} \| \text{deps}(t)\big) \quad (5)$$

其中，sk_s 為伺服器的私鑰簽名鍵，H 為抗碰撞的雜湊函數，$\text{deps}(t)$ 為該工具依賴樹的雜湊值。

L-CTA 層維護一份證明日誌（類似於憑證透明度），記錄所有工具證明記錄。在每次調用之前：

1) 驗證 $\text{TAR}(t)$ 與伺服器公開金鑰；
2) 將 $H(\text{def}_{\text{current}}(t))$ 與核准的 hash 進行比對；
3) 驗證版本 (t) 未出現回退（防止 TV6);
4) 檢查 $\text{deps}(t)$ 與已知良好的依賴哈希值 (防範 TV16–TV17)。

D. Layer 3: Information Flow Tracking

L-IFT enforces data confinement (Property 2) through dynamic taint tracking at the MCP message level. Each piece of data entering the agent's context from a tool response is labeled with its originating trust domain:

Definition 6 (Tainted Data)**.** *A tainted datum is a pair* (d, ℓ_d) *where* d *is the data value and* $\ell_d \in \mathcal{L}$ *is its security label, inherited from the trust domain of the MCP server that produced it.*

When the agent composes a new tool invocation, L-IFT checks that no argument contains data with a security label higher than the target server's clearance level:

$$\forall \text{arg} \in \text{args}(\texttt{call}(t)) : \quad \ell(\text{arg}) \sqsubseteq \ell(\mathcal{D}_{\text{server}(t)}) \quad (6)$$

This directly defends against cross-server data leakage (TC3). For context isolation (Property 4), L-IFT additionally tracks information provenance through the LLM's reasoning steps, flagging any case where data from server s_i influences a request to server s_j without authorization.

E. Layer 4: Runtime Policy Enforcement

L-RPE implements a security automaton [33] that monitors the MCP transition system in real time. We adopt the edit automaton model [34], which can suppress, insert, or modify actions in the event stream.

Definition 7 (MCP Security Automaton)**.** *An MCP security automaton is a tuple*

$$\mathcal{E} = (Q, q_0, \Lambda, \delta, \gamma)$$

where Q *is a finite set of monitor states,* q_0 *is the initial state,* Λ *is the MCP action alphabet,* $\delta \colon Q \times \Lambda \to Q$ *is the transition function, and* $\gamma \colon Q \times \Lambda \to \{\mathit{allow}, \mathit{suppress}, \mathit{modify}(\lambda')\}$ *is the enforcement function.*

The security automaton encodes policies such as:

- **Rate limiting**: Suppress tool invocations exceeding a threshold within a time window (defends against denial-of-service via excessive tool calls).
- **Anomaly detection**: Flag unusual sequences of tool invocations that deviate from learned behavioral profiles (defends against TV12, TV13).
- **Consent enforcement**: Insert user confirmation requests before high-risk operations, even if the agent attempts to auto-approve (defends against TV13, TV14).
- **Semantic validation**: Modify tool invocation arguments that contain suspected prompt injection patterns, stripping adversarial content before it reaches the MCP server (defends against TC1, TC6).

F. Integrated Coverage

Table III shows how MCPSHIELD's four layers complement each other to achieve comprehensive coverage.

The combined architecture achieves 91% theoretical coverage (21 of 23 attack vectors fully addressed). The two remaining vectors—TV10 (channel coercion) and TV19 (memory poisoning in long-term agent memory)—require environment-level controls and LLM-internal mechanisms, respectively, which fall outside the protocol-level scope of MCPSHIELD.

VII. Discussion

A. Implications for MCP Protocol Design

Our analysis reveals that several threat categories arise from fundamental design decisions in the MCP specification. Tool descriptions are consumed as natural language by LLMs, creating an inherent prompt injection surface (TC1, TC6). The current specification does not mandate tool definition immutability, enabling rug pulls (TC2). Session management lacks built-in cryptographic protection, exposing transport-level vulnerabilities (TC7). We recommend that the AAIF incorporate the following into future MCP specification revisions:

1) **Mandatory tool attestation**: Tool definitions must carry cryptographic signatures that clients verify before presentation to the LLM.
2) **Structured tool descriptions**: Replace free-form natural language descriptions with a structured schema that separates machine-parseable functionality specifications from human-readable documentation, reducing the prompt injection surface.
3) **Built-in session security**: Integrate message signing and nonce-based replay protection as mandatory protocol features, not optional extensions.

B. The Composition Challenge

The most novel threat surface identified in our taxonomy is $\mathcal{S}_{\text{compose}}$—the emergent attack surface from multi-server, multi-tool, and cross-protocol interactions. Traditional security frameworks analyze components in isolation, but MCP-based agent systems exhibit emergent risks that only manifest when tools are composed. Capability chaining (TV12) and cross-protocol confusion (TV23) exemplify this: each component may be individually secure, yet their composition is not.

This mirrors the well-known *confused deputy* problem [26] but at a protocol level. Our composition policies in L-CAC provide a first step, but a complete solution requires *compositional security proofs*—showing that security properties of individual components are preserved under composition. This remains an open theoretical challenge.

C. Scalability and Performance Considerations

MCPSHIELD's four-layer architecture introduces overhead at each tool invocation. Based on existing component benchmarks:

- **L-CAC**: Capability verification is $O(1)$ with pre-computed capability sets. Estimated overhead: <1ms per invocation.
- **L-CTA**: Signature verification (ECDSA P-256) requires approximately 0.5ms. Hash comparison is negligible. Estimated overhead: <2ms per invocation.

D. 第 3 層：資訊流程追蹤

L-IFT 透過在 MCP 訊息層級執行動態污點追蹤，強制執行資料封閉（Property2）。每個從工具回應進入代理程式上下文的資料片段，都會被標記其來源的信任領域：

定義 6（受污染資料）。受污染資料是一對 (d, ℓ_d)，其中 d 爲資料值，$\ell_d \in \mathcal{L}$ 爲其安全標籤，該標籤繼承自產生該資料的 *MCP* 伺服器的信任領域。

當代理組成新的工具調用時，L-IFT 檢查是否有任何參數包含安全標記高於目標伺服器清權等級的資料：

$$\forall \text{arg} \in \text{args}(\text{call}(t)) : \quad \ell(\text{arg}) \sqsubseteq \ell(\mathcal{D}_{\text{server}(t)}) \quad (6)$$

這直接防禦跨伺服器資料洩漏（TC3）。就資料隔離（屬性 4）而言，L-IFT 額外透過追蹤 LLM 的推理步驟來記錄資訊來源，並標記任何未經授權情況下，來自伺服器 s_i 的資料影響對伺服器 s_j 的請求的情況。

E. 第 4 層：執行時策略執行

L-RPE 實現了一個安全自動機 [33]，可實時監控 MCP 遷移系統。我們採用編輯自動機模型 [34]，該模型能夠抑制、插入或修改事件流中的動作。

定義 7（*MCP* 安全自動機）。*MCP* 安全自動機是一個元組

$$\mathcal{E} = (Q, q_0, \Lambda, \delta, \gamma)$$

其中 Q 爲監控狀態的有限集合，q_0 爲初始狀態，Λ 爲 *MCP* 動作字母表，$\delta \colon Q \times \Lambda \to Q$ 爲遷移函數，且 $\gamma \colon Q \times \Lambda \to \{$ 允許、抑制、修改 $(\lambda')\}$ 爲執行函數。

安全機器人編碼如下的策略：

- 速率限制：在指定時間窗口內抑制超過閾值的工具調用（防止因過度調用工具而引發服務中斷）。
- 異常檢測：標記與已學習的行為模式不符的工具調用異常序列（防禦 TV12、TV13）。
- 同意執行：在高風險操作前插入使用者確認請求，即使代理嘗試自動批准（可防禦 TV13、TV14）。
- 語義驗證：修改工具調用參數，使包含疑似提示注入模式的內容，在內容傳送至 MCP 伺服器前會移除對抗性內容（防範 TC1、TC6）。

F. 整合覆蓋

表 III 顯示 MCPSHIELD 的四層如何相互補充以實現全面覆蓋。

整合後的架構實現了 91% 的理論覆蓋率（23 個攻擊向量中的 21 個已完全解決）。剩餘的兩個向量——TV10（頻道強制）和 TV19（長期代理記憶中的記憶污染）——分別需要環境層控制與 LLM 內部機制，這超出了 MCPSHIELD 的協議層範圍。

VII. 討論

A. 對 *MCP* 協議設計的啟示

我們的分析顯示，多個威脅類別源於 MCP 規範中的基本設計決策。工具描述被 LLM 以自然語言消費，這構成了本質性的提示注入風險面（TC1, TC6）。當前的規範未強制要求工具定義的不可變性，從而使 rug pulls 成為可能（TC2）。會話管理缺乏內建的加密保護，暴露了傳輸層漏洞（TC7）。我們建議 AAIF 將以下內容納入未來 MCP 規範的修訂中：

1) 強制工具認證：工具定義必須包含客戶端在展示前驗證的加密簽名，且
對 LLM 的調用 .
2) 結構化工具說明：將自由形式的自然語言描述替換為結構化模式，將機器可解析的功能規範與人類可讀的說明文檔分開，從而降低提示注入風險。

3) 內建會話安全性：將訊息簽名與基於非重複碼的防重放保護整合為強制性協議功能，而非可選擴充功能。

B. 組成挑戰

我們分類法中識別出的最具創新性的威脅面是 $\mathcal{S}_{\text{compose}}$——源自多伺服器、多工具及跨協議互動的新興攻擊面。傳統安全架構會孤立分析各個組件，但基於 MCP 的代理系統卻會展現出僅在工具組合時才會顯現的新興風險。能力鏈接（TV12）與跨協議混淆（TV23）便是此類範例：每個組件可能各自安全，但其組合卻並非如此。

這反映了眾所周知的「混亂代理人」問題 [26]，但是在協議層面。我們的 L-CAC 組件策略提供了一個初步步驟，但完整的解決方案需要組件安全性證明——即證明在組件組合下，各個組件的安全屬性得以保留。這仍是一個未解決的理論挑戰。

C. 可擴展性與效能考量

MCPSHIELD 的四層架構在每次工具調用時會引入額外開銷。基於現有的組件基準測試：

- L-CAC：能力驗證為 $O(1)$ 使用預先計算的能力集。預估過載：<1ms 每次調用。
- L-CTA：ECDSA P-256 簽名驗證約需 0.5ms，雜湊比對可忽略不計。Es-
預估的額外開銷：<2ms 每次呼叫。

TABLE III
MCPSHIELD LAYER COVERAGE ACROSS THREAT CATEGORIES.

Layer	TC1	TC2	TC3	TC4	TC5	TC6	TC7	Cov.
L-CAC	◐	·	·	●	·	·	·	17%
L-CTA	●	●	·	·	●	·	◐	39%
L-IFT	·	·	●	◐	·	◐	·	22%
L-RPE	●	◐	◐	●	◐	●	●	70%
Combined	●	●	●	●	●	●	●	**91%**

- **L-IFT**: Taint propagation at the message level (not token level) adds minimal overhead. Estimated overhead: <1ms per invocation.
- **L-RPE**: MCP-Guard reports <2ms for Stage 1 pattern matching [15]. Security automaton state transitions are $O(1)$. Estimated overhead: 2–5ms per invocation.

Total estimated overhead of 4–9ms per tool invocation is negligible relative to LLM inference latency (typically 500ms–5s) and network round-trip times.

D. Limitations

Our work has several limitations that we acknowledge transparently:

1) **Theoretical framework without implementation.** MCPSHIELD is a reference architecture with formal properties but has not been implemented or empirically evaluated. Real-world deployment may reveal practical challenges not captured in our model.
2) **LLM-internal attacks are out of scope.** Our formal model operates at the protocol level and cannot address attacks that exploit LLM-internal reasoning (e.g., sophisticated prompt injections that bypass semantic analysis). Defense against these requires advances in LLM alignment and robustness.
3) **Coverage metric is theoretical.** Our 91% coverage claim is based on mapping defense layers to threat vectors. Actual effectiveness depends on the quality of implementation and the adversary's sophistication.
4) **Dynamic threat landscape.** Our taxonomy reflects the state of MCP security research as of early 2026. New attack vectors will emerge as the ecosystem evolves.
5) **Cross-protocol security is nascent.** Our treatment of A2A/ACP/ANP interoperability threats (TV23) is preliminary, as these protocols are still maturing.

VIII. OPEN RESEARCH CHALLENGES

Our systematization identifies seven research challenges that must be addressed to secure MCP-based AI agents at scale.

Challenge 1: Compositional Security Proofs. Proving that the composition of individually secure MCP tools preserves security properties is an open theoretical problem. Existing approaches from protocol composition (e.g., the UC framework) do not directly apply because MCP tools are not traditional cryptographic protocols—they involve natural language interfaces and non-deterministic LLM behavior.

Challenge 2: Semantic Integrity Verification. Cryptographic signatures verify syntactic integrity (the tool definition has not changed) but not semantic integrity (the tool still does what it claims). A tool could maintain an identical description while modifying its server-side implementation. Verifying semantic integrity likely requires reproducible builds, formal specifications of tool behavior, and runtime behavioral monitoring.

Challenge 3: Scalable Information Flow Tracking. Our L-IFT layer tracks information at the message level, but LLMs transform data through attention mechanisms that make fine-grained provenance tracking intractable. Developing scalable abstractions for information flow through LLMs—perhaps leveraging influence functions or activation analysis—is an open problem.

Challenge 4: Adversarial Robustness of Defense Layers. Each MCPSHIELD layer can itself become an attack target. An adversary might craft tool descriptions that evade MCP-Guard's pattern matching, forge capabilities by exploiting implementation bugs, or poison the behavioral profiles used for anomaly detection. A security analysis of the defense architecture itself (defense-against-defense attacks) is needed.

Challenge 5: Cross-Protocol Trust Federation. As agents increasingly bridge MCP, A2A, ACP, and ANP, a federated trust framework is needed that can translate and compose trust assertions across heterogeneous protocols. The current landscape of 4+ competing protocols [19] without interoperable trust semantics is a significant gap.

Challenge 6: Dynamic Consent and Human-in-the-Loop. Current consent models are binary (approve/deny) and static (granted at connection time). Real-world MCP usage requires dynamic, context-sensitive consent that adapts to the risk level of each specific invocation. Designing consent UIs and policies that balance security with usability in real-time agentic workflows is an HCI and security co-design challenge.

Challenge 7: Ecosystem-Level Governance. With 177,000+ tools and growing [7], the MCP ecosystem faces the same governance challenges as npm, PyPI, and Docker Hub, but with additional LLM-specific risks. Nguyen et al. [38] found that "Non-Determinism" and "Data Leakage" are the most inadequately addressed risk domains across 16 existing security frameworks. The AAIF [8] provides governance structure, but technical mechanisms for server reputation, tool certification, and ecosystem-wide threat intelligence remain undeveloped.

表 III 顯示 MCPSHIELD 層在不同威脅類別中的覆蓋範圍。

層	TC1	TC2	TC3	TC4	TC5	TC6	TC7	Cov.
L-CAC	◐	·	·	●	·	·	·	17%
L-CTA	●	●	·	·	●	·	◐	39%
L-IFT	·	·	●	◐	·	◐	·	22%
L-RPE	●	◐	◐	●	◐	●	●	70%
合併	●	●	●	●	●	●	●	**91%**

• L-IFT：在訊息層面（而非 token 層面）進行污點傳播，僅增加極小的負載。預估負載： <1ms 每次呼叫。

•L-RPE: MCP-Guard 報告 <2ms 用於第 1 階段模式匹配 [15]。安全自動化機態轉換為$O(1)$。預估過載：每次調用 2–5ms。

每工具調用預估的總過渡時間為 4–9 毫秒
相對於 LLM 推論延遲（通常為 500ms–5s）及網路往返時間而言，可忽略不計。

D. 限制

我們的工作存在若干限制，我們坦誠地承認：

1) 僅有理論框架，尚未實施。MCP-SHIELD 是一個具有形式化特性的參考架構，但尚未實施或進行經驗驗證。實際部署可能會揭示我們的模型未能捕捉到的實際挑戰。

2) LLM 內部攻擊不在範圍內。我們的形式模型運作於協議層級，無法處理利用 LLM 內部推理的攻擊（例如，能繞過語義分析的複雜提示注入）。針對這些攻擊的防禦需要 LLM 對齊與韌性的進展。

3) 覆蓋率指標為理論值。我們所宣稱的 91% 覆蓋率是基於將防禦層映射至威脅向量而得出。實際效能取決於實施品質與攻擊者的技術水準。

4) 動態威脅環境。我們的分類法反映了截至 2026 年初 MCP 安全研究的現狀。隨著生態系統的演變，新的攻擊向量將不斷出現。

5) 跨協議安全尚處於初創階段。我們的處理方 f
A2A/ACP/ANP 互操作性威脅 (TV23) 目前僅為初步階段，因為這些協議仍在成熟中。

VIII. 開放式研究挑戰

我們的系統化方法識別出七項必須解決的研究挑戰，以確保大規模部署基於 MCP 的 AI 代理。

挑戰 1：組件安全性證明。證明個別安全的 MCP 工具組合後仍能保持安全性屬性，是一個尚未解決的理論問題。現有的協議組件方法（例如 uC 框架）無法直接套用，因為 MCP 工具並非傳統密碼學協議——它們涉及自然語言介面以及非確定性的大語言模型行為。

挑戰 2：語義完整性驗證。密碼學數位簽章可驗證語法完整性（工具定義未改變），但無法驗證語義完整性（工具仍如其所宣稱般運作）。工具可能維持相同的描述，同時修改其伺服器端實作。驗證語義完整性可能需要可重複的建置、工具行為的形式化規範，以及執行時行為監控。

挑戰 3：可擴展的資訊流追蹤。我們的 L-IFT 層在訊息層級追蹤資訊，但大型語言模型透過注意力機制處理資料，使得細緻的來源追蹤變得難以實現。開發用於追蹤大型語言模型中資訊流的可擴展抽象方法——或許利用影響函數或激活分析——仍是一個未解之題。

挑戰 4：防禦層的對抗魯棒性。每個 MCPSHIELD 層本身都可能成為攻擊目標。攻擊者可能會編製能繞過 MCP-Guard 模式匹配的工具描述，透過利用實現漏洞偽造能力，或污染用於異常檢測的行為特徵。因此，必須對防禦架構本身進行安全分析（即防禦對抗防禦的攻擊）。

挑戰 5：跨協議信任聯盟。隨著代理越來越頻繁地連接 MCP、A2A、ACP 和 ANP，一個聯盟式的
需要一個信任框架，能夠跨異質協議進行信任斷言的翻譯與組合。目前 4+ 競爭協議 [19] 缺乏可互操作信任語義的現狀，是一個重大缺口。

挑戰 6：動態同意與人在迴路。目前的同意模式是二元的（批准/拒絕）且靜態的（在連線時授予）。現實世界中的 MCP 使用需要動態的、情境敏感的同意機制，能根據每次特定調用的風險水平進行調整。設計能夠在即時代理工作流中平衡安全性與可用性的同意介面與政策，是HCI與安全共同設計的挑戰。

挑戰 7：生態層級治理。隨著擁有 177,000+ 工具且持續增長 [7]，MCP 生態系統面臨與 npm、PyPI 和 Docker Hub 相同的治理挑戰，但還需應對額外的 LLM 特定風險。Nguyen 等 [38] 發現，「非確定性」與「資料洩漏」是現有 16 個安全框架中處理最不足的風險領域。AAIF [8] 提供了治理架構，但關於伺服器聲譽、工具認證及生態系統層面威脅智取的技術機制仍尚未發展。

IX. Conclusion

This paper presented MCPSHIELD, a formal security framework for MCP-based AI agents that addresses the critical gap of fragmented security research in the rapidly growing MCP ecosystem. Through systematic analysis of 12 MCP security papers, 5 benchmarks, and the 177,000-tool empirical landscape, we constructed a unified threat taxonomy of 7 categories and 23 attack vectors across 4 attack surfaces. Our formal verification model, based on labeled transition systems with trust-boundary annotations, defines four fundamental security properties—tool integrity, data confinement, privilege boundedness, and context isolation—with decidability results for their verification. Our comparative evaluation revealed that no existing defense mechanism covers more than 34% of the threat landscape, whereas MCPSHIELD's integrated four-layer architecture achieves 91% theoretical coverage.

As MCP transitions from an Anthropic project to vendor-neutral infrastructure under the Linux Foundation, and as action-capable tools grow from 27% to 65% of the ecosystem, the security stakes have never been higher. The International AI Safety Report 2026 [39] concludes that AI risk management techniques are "improving but insufficient." We hope that MCPSHIELD's formal foundations, systematic analysis, and identified research challenges will catalyze the next generation of MCP security research and guide the AAIF in developing enforceable security standards for the agentic AI era.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proceedings of the International Conference on Learning Representations*, 2023.

[2] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[3] M. Abou Ali and F. Dornaika, "Agentic AI: A comprehensive survey of architectures, applications, and future directions," *Artificial Intelligence Review*, 2025.

[4] Z. Deng, Y. Guo, C. Han, W. Ma, J. Xiong, S. Wen, and Y. Xiang, "AI agents under threat: A survey of key security challenges and future pathways," *ACM Computing Surveys*, 2025.

[5] Anthropic, "Introducing the Model Context Protocol," https://www.anthropic.com/news/model-context-protocol, 2024, accessed: 2026-04-01.

[6] ——, "Model Context Protocol specification (version 2025-11-25)," https://modelcontextprotocol.io/specification/2025-11-25, 2025, accessed: 2026-04-01.

[7] M. Stein, "How are AI agents used? Evidence from 177,000 MCP tools," arXiv preprint arXiv:2603.23802, 2026.

[8] Linux Foundation, "Linux Foundation announces the formation of the Agentic AI Foundation," https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundat 2025, accessed: 2026-04-01.

[9] Invariant Labs, "MCP security notification: Tool poisoning attacks," https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks, 2025, accessed: 2026-04-01.

[10] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, and X. Li, "MCPTox: A benchmark for tool poisoning attack on real-world MCP servers," arXiv preprint arXiv:2508.14925, 2025.

[11] M. Bhatt, V. S. Narajala, and I. Habler, "ETDI: Mitigating tool squatting and rug pull attacks in Model Context Protocol (MCP) by using OAuth-enhanced tool definitions and policy-based access control," arXiv preprint arXiv:2506.01333, 2025.

[12] Anonymous, "Log-to-leak: Prompt injection attacks on tool-using LLM agents via Model Context Protocol," OpenReview, 2025, available at https://openreview.net/forum?id=UVgbFuXPaO.

[13] Y. Yang, C. Gao, D. Wu, Y. Chen, Y. Li, and S. Wang, "MCPSecBench: A systematic security benchmark and playground for testing Model Context Protocols," arXiv preprint arXiv:2508.13220, 2025.

[14] X. Zong, Z. Shen, L. Wang, Y. Lan, and C. Yang, "MCP-SafetyBench: A benchmark for safety evaluation of large language models with real-world MCP servers," arXiv preprint arXiv:2512.15163, 2025.

[15] W. Xing, Z. Qi, Y. Qin, Y. Li, C. Chang, J. Yu, C. Lin, Z. Xie, and M. Han, "MCP-Guard: A multi-stage defense-in-depth framework for securing Model Context Protocol in agentic AI," arXiv preprint arXiv:2508.10991, 2025.

[16] B. Wang, Z. Liu, H. Yu, A. Yang, Y. Huang, J. Guo, H. Cheng, H. Li, and H. Wu, "MCPGuard: Automatically detecting vulnerabilities in MCP servers," arXiv preprint arXiv:2510.23673, 2025.

[17] V. S. Narajala and I. Habler, "Enterprise-grade security for the Model Context Protocol (MCP): Frameworks and mitigation strategies," arXiv preprint arXiv:2504.08623, 2025.

[18] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model Context Protocol (MCP): Landscape, security threats, and future research directions," *ACM Transactions on Software Engineering and Methodology*, 2026.

[19] A. Ehtesham, A. Singh, G. K. Gupta, and S. Kumar, "A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)," arXiv preprint arXiv:2505.02279, 2025.

[20] Google, "Agent2Agent Protocol (A2A)," https://a2a-protocol.org/latest/, 2025, accessed: 2026-04-01.

[21] S. Jamshidi, K. W. Nafi, A. M. Dakhel, N. Shahabi, F. Khomh, and N. Ezzati-Jivan, "Securing the Model Context Protocol: Defending LLMs against tool poisoning and adversarial attacks," arXiv preprint arXiv:2512.06556, 2025.

[22] S. Jamshidi, K. W. Nafi, A. M. Dakhel, F. Khomh,

10

IX. 結論

本文提出了 MCPSHIELD，一個針對基於 MCP 的 AI 代理的正式安全框架，旨在解決快速增長的 MCP 生態系統中碎片化安全研究所帶來的關鍵缺口。透過對 12 篇 MCP 安全論文、5 個基準測試以及 177,000 個工具的實證景況進行系統性分析，我們構建了一個涵蓋 4 個攻擊面、包含 7 個類別與 23 種攻擊向量的統一威脅分類法。我們的正式驗證模型基於帶有信任邊界註釋的標記過渡系統，定義了四項基本安全屬性——工具完整性、資料封閉性、權限有界性與情境隔離，並提供了其驗證的可判定性結果。我們的比較評估顯示，現有的防禦機制無法覆蓋超過 34% 的威脅景況，而 MCPSHIELD 的整合四層架構則實現了 91% 的理論覆蓋率。

隨著 MCP 從 Anthropic 專案轉變為由 Linux Foundation 管理的無供應商中立基礎設施，且具備行動能力的工具佔生態系比例從 27% 增長至 65%，安全風險從未如此高企。《國際 AI 安全報告 2026 [39]》結論指出，AI 風險管理技術「正在改善但仍顯不足」。我們希望 MCPSHIELD 的正式基礎、系統性分析以及已識別的研究挑戰，能催化下一代 MCP 安全研究，並引導 AAIF 為代理式 AI 時代制定可執行的安全標準。

參考資料

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proceedings of the International Conference on Learning Representations*, 2023.

[2] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[3] M. Abou Ali and F. Dornaika, "Agentic AI: A comprehensive survey of architectures, applications, and future directions," *Artificial Intelligence Review*, 2025.

[4] Z. Deng, Y. Guo, C. Han, W. Ma, J. Xiong, S. Wen, and Y. Xiang, "AI agents under threat: A survey of key security challenges and future pathways," *ACM Computing Surveys*, 2025.

[5] Anthropic, "Introducing the Model Context Protocol," https://www.anthropic.com/news/model-context-protocol, 2024, accessed: 2026-04-01.

[6] ——, "Model Context Protocol specification (version 2025-11-25)," https://modelcontextprotocol.io/specification/2025-11-25, 2025, accessed: 2026-04-01.

[7] M. Stein, "How are AI agents used? Evidence from 177,000 MCP tools," arXiv preprint arXiv:2603.23802, 2026.

[8] Linux Foundation, "Linux Foundation announces the formation of the Agentic AI Foundation," https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundat 2025, accessed: 2026-04-01.

[9] Invariant Labs, "MCP security notification: Tool poisoning attacks," https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks, 2025, accessed: 2026-04-01.

[10] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, and X. Li, "MCPTox: A benchmark for tool poisoning attack on real-world MCP servers," arXiv preprint arXiv:2508.14925, 2025.

[11] M. Bhatt, V. S. Narajala, and I. Habler, "ETDI: Mitigating tool squatting and rug pull attacks in Model Context Protocol (MCP) by using OAuth-enhanced tool definitions and policy-based access control," arXiv preprint arXiv:2506.01333, 2025.

[12] Anonymous, "Log-to-leak: Prompt injection attacks on tool-using LLM agents via Model Context Protocol," OpenReview, 2025, available at https://openreview.net/forum?id=UVgbFuXPaO.

[13] Y. Yang, C. Gao, D. Wu, Y. Chen, Y. Li, and S. Wang, "MCPSecBench: A systematic security benchmark and playground for testing Model Context Protocols," arXiv preprint arXiv:2508.13220, 2025.

[14] X. Zong, Z. Shen, L. Wang, Y. Lan, and C. Yang, "MCP-SafetyBench: A benchmark for safety evaluation of large language models with real-world MCP servers," arXiv preprint arXiv:2512.15163, 2025.

[15] W. Xing, Z. Qi, Y. Qin, Y. Li, C. Chang, J. Yu, C. Lin, Z. Xie, and M. Han, "MCP-Guard: A multi-stage defense-in-depth framework for securing Model Context Protocol in agentic AI," arXiv preprint arXiv:2508.10991, 2025.

[16] B. Wang, Z. Liu, H. Yu, A. Yang, Y. Huang, J. Guo, H. Cheng, H. Li, and H. Wu, "MCPGuard: Automatically detecting vulnerabilities in MCP servers," arXiv preprint arXiv:2510.23673, 2025.

[17] V. S. Narajala and I. Habler, "Enterprise-grade security for the Model Context Protocol (MCP): Frameworks and mitigation strategies," arXiv preprint arXiv:2504.08623, 2025.

[18] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model Context Protocol (MCP): Landscape, security threats, and future research directions," *ACM Transactions on Software Engineering and Methodology*, 2026.

[19] A. Ehtesham, A. Singh, G. K. Gupta, and S. Kumar, "A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)," arXiv preprint arXiv:2505.02279, 2025.

[20] Google, "Agent2Agent Protocol (A2A)," https://a2a-protocol.org/latest/, 2025, accessed: 2026-04-01.

[21] S. Jamshidi, K. W. Nafi, A. M. Dakhel, N. Shahabi, F. Khomh, and N. Ezzati-Jivan, "Securing the Model Context Protocol: Defending LLMs against tool poisoning and adversarial attacks," arXiv preprint arXiv:2512.06556, 2025.

[22] S. Jamshidi, K. W. Nafi, A. M. Dakhel, F. Khomh,

A. Nikanjam, and M. A. Hamdaqa, "Secure tool manifest and digital signing solution for verifiable MCP and LLM pipelines," arXiv preprint arXiv:2601.23132, 2026.
[23] R. Sharif, "MCPS — MCP Secure: Cryptographic identity, message signing, and trust verification for the Model Context Protocol," https://mcp-secure.dev/, 2026, iETF Internet-Draft draft-sharif-mcps-secure-mcp-00. Accessed: 2026-04-01.
[24] F. He, T. Zhu, D. Ye, B. Liu, W. Zhou, and P. S. Yu, "The emerged security and privacy of LLM agent: A survey with case studies," *ACM Computing Surveys*, 2025.
[25] Y. Tang, Y. Liu, J. Lan, Z. Yan, and E. Gelenbe, "Security of LLM-based agents regarding attacks, defenses, and applications: A comprehensive survey," *Information Fusion*, vol. 127, 2026.
[26] A. Dehghantanha and S. Homayoun, "SoK: The attack surface of agentic AI — tools, and autonomy," arXiv preprint arXiv:2603.22928, 2026.
[27] S. Wilson and A. Dawson, "OWASP top 10 for large language model applications (version 2025)," https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/, 2025, accessed: 2026-04-01.
[28] A. Shostack, *Threat Modeling: Designing for Security*. Wiley, 2014.
[29] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "SoK: Taxonomy of attacks on open-source software supply chains," in *Proceedings of the IEEE Symposium on Security and Privacy*. IEEE, 2023.
[30] S. Gulyamov, S. Gulyamov, A. Rodionov, R. Khursanov, K. Mekhmonov, D. Babaev, and A. Rakhimjonov, "Prompt injection attacks in large language models and AI agent systems: A comprehensive review of vulnerabilities, attack vectors, and defense mechanisms," *Information*, vol. 17, no. 1, p. 54, 2026.
[31] M. A. Ferrag, N. Tihanyi, D. Hamouda, L. Maglaras, A. Lakas, and M. Debbah, "From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows," *ICT Express*, 2025.
[32] D. E. Denning, "A lattice model of secure information flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976.
[33] F. B. Schneider, "Enforceable security policies," *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, 2000.
[34] J. Ligatti, L. Bauer, and D. Walker, "Edit automata: Enforcement mechanisms for run-time security policies," *International Journal of Information Security*, vol. 4, no. 1–2, pp. 2–16, 2005.
[35] Microsoft, "Introducing the Agent Governance Toolkit: Open-source runtime security for AI agents," https://opensource.microsoft.com/blog/2026/04/02/introducing-the-agent-governance-toolkit-open-source-runtime-security-for-ai-agents/, 2026, accessed: 2026-04-04.
[36] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," National Institute of Standards and Technology, Tech. Rep. SP 800-207, 2020.
[37] J. B. Dennis and E. C. Van Horn, "Programming semantics for multiprogrammed computations," *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966.
[38] T. Nguyen, M. Ndebugre, and D. Arremsetty, "Security considerations for multi-agent systems," arXiv preprint arXiv:2603.09002, 2026.
[39] Y. Bengio *et al.*, "International AI safety report 2026," arXiv preprint arXiv:2602.21012, 2026.

11

A. Nikanjam, and M. A. Hamdaqa, "Secure tool manifest and digital signing solution for verifiable MCP and LLM pipelines," arXiv preprint arXiv:2601.23132, 2026.
[23] R. Sharif, "MCPS — MCP Secure: Cryptographic identity, message signing, and trust verification for the Model Context Protocol," https://mcp-secure.dev/, 2026, iETF Internet-Draft draft-sharif-mcps-secure-mcp-00. Accessed: 2026-04-01.
[24] F. He, T. Zhu, D. Ye, B. Liu, W. Zhou, and P. S. Yu, "The emerged security and privacy of LLM agent: A survey with case studies," *ACM Computing Surveys*, 2025.
[25] Y. Tang, Y. Liu, J. Lan, Z. Yan, and E. Gelenbe, "Security of LLM-based agents regarding attacks, defenses, and applications: A comprehensive survey," *Information Fusion*, vol. 127, 2026.
[26] A. Dehghantanha and S. Homayoun, "SoK: The attack surface of agentic AI — tools, and autonomy," arXiv preprint arXiv:2603.22928, 2026.
[27] S. Wilson and A. Dawson, "OWASP top 10 for large language model applications (version 2025)," https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/, 2025, accessed: 2026-04-01.
[28] A. Shostack, *Threat Modeling: Designing for Security*. Wiley, 2014.
[29] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "SoK: Taxonomy of attacks on open-source software supply chains," in *Proceedings of the IEEE Symposium on Security and Privacy*. IEEE, 2023.
[30] S. Gulyamov, S. Gulyamov, A. Rodionov, R. Khursanov, K. Mekhmonov, D. Babaev, and A. Rakhimjonov, "Prompt injection attacks in large language models and AI agent systems: A comprehensive review of vulnerabilities, attack vectors, and defense mechanisms," *Information*, vol. 17, no. 1, p. 54, 2026.
[31] M. A. Ferrag, N. Tihanyi, D. Hamouda, L. Maglaras, A. Lakas, and M. Debbah, "From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows," *ICT Express*, 2025.
[32] D. E. Denning, "A lattice model of secure information flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976.
[33] F. B. Schneider, "Enforceable security policies," *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, 2000.
[34] J. Ligatti, L. Bauer, and D. Walker, "Edit automata: Enforcement mechanisms for run-time security policies," *International Journal of Information Security*, vol. 4, no. 1–2, pp. 2–16, 2005.
[35] Microsoft, "Introducing the Agent Governance Toolkit: Open-source runtime security for AI agents," https://opensource.microsoft.com/blog/2026/04/02/introducing-the-agent-governance-toolkit-open-source-runtime-security-for-ai-agents/, 2026, accessed: 2026-04-04.
[36] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," National Institute of Standards and Technology, Tech. Rep. SP 800-207, 2020.
[37] J. B. Dennis and E. C. Van Horn, "Programming semantics for multiprogrammed computations," *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966.
[38] T. Nguyen, M. Ndebugre, and D. Arremsetty, "Security considerations for multi-agent systems," arXiv preprint arXiv:2603.09002, 2026.
[39] Y. Bengio *et al.*, "International AI safety report 2026," arXiv preprint arXiv:2602.21012, 2026.